Intelligence artificielle

Redha Moulla

BRGM, Orléans, 19 – 21 octobre 2026

Plan

- Introduction à l'intelligence artificielle
- Introduction au NLP
- Techniques statistiques pour le NLP
- Machine learning pour le NLP
- Deep learning pour le NLP
- Transformers et LLMs
- Enjeux éthiques

Qu'est-ce que l'intelligence artificielle ?

Définition littérale de l'intelligence artificielle

1. Intelligence

Ensemble des fonctions mentales ayant pour objet la connaissance conceptuelle et relationnelle.

- Larousse

2. Artificielle

Qui est produit de l'activité humaine (opposé à la nature).

- Larousse

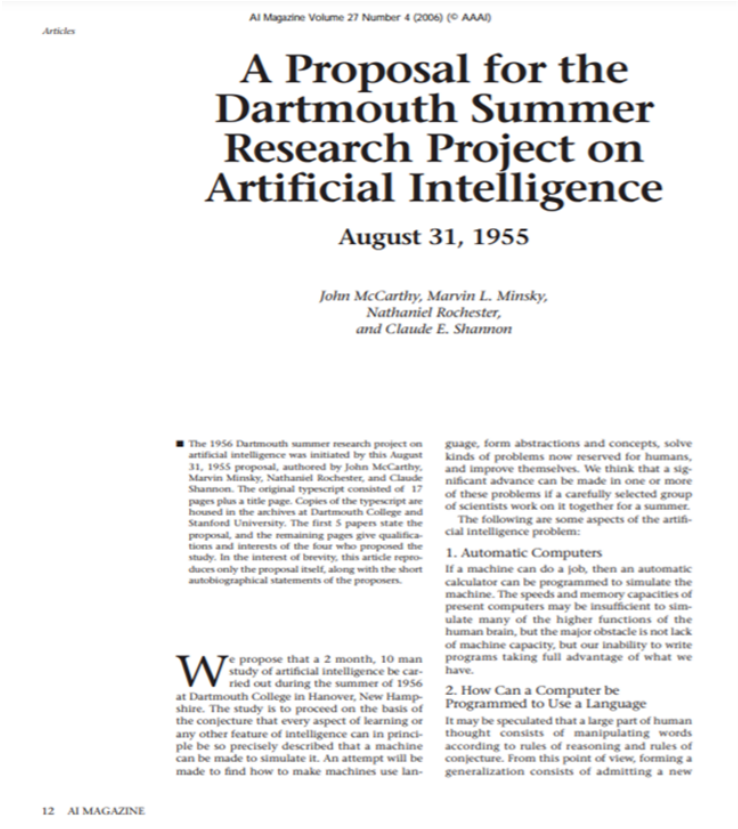
Qu'est-ce que l'intelligence ?

La notion d'intelligence recouvre plusieurs facultés cognitives :

1. **Raisonnement** : La capacité à résoudre des problèmes et à faire des déductions logiques.
2. **Apprentissage** : L'aptitude à acquérir de nouvelles connaissances et à s'améliorer grâce à l'expérience.
3. **Perception** : La compétence pour reconnaître et interpréter les stimuli sensoriels.
4. **Compréhension** : L'habileté à saisir le sens et l'importance de divers concepts et situations.
5. **Mémorisation** : La faculté de stocker et de rappeler des informations.
6. **Créativité** : Le pouvoir d'inventer ou de produire de nouvelles idées, de l'originalité dans la pensée.

Mais est-ce que l'intelligence est réductible à des facultés mesurables ?

Conférence de Dartmouth ?



Articles

AI Magazine Volume 27 Number 4 (© AAAI)

A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence

August 31, 1955

John McCarthy, Marvin L. Minsky, Nathaniel Rochester, and Claude E. Shannon

■ The 1956 Dartmouth summer research project on artificial intelligence was initiated by this August 31, 1955 proposal, authored by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon. The original typescript consisted of 17 pages plus a title page. Copies of the typescript are housed in the archives at Dartmouth College and Stanford University. The first 5 papers state the proposal, and the remaining pages give qualifications and interests of the four who proposed the study. In the interest of brevity, this article repeoducers only the proposal itself, along with the short autobiographical statements of the proposers.

We propose that a 2 month, 10 man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer.

The following are some aspects of the artificial intelligence problem:

1. Automatic Computers

If a machine can do a job, then an automatic calculator can be programmed to simulate the machine. The speeds and memory capacities of present computers may be insufficient to simulate many of the higher functions of the human brain, but the major obstacle is not lack of machine capacity, but our inability to write programs taking full advantage of what we have.

2. How Can a Computer be Programmed to Use a Language

It may be speculated that a large part of human thought consists of manipulating words according to rules of reasoning and rules of conjecture. From this point of view, forming a generalization consists of admitting a new

12 AI MAGAZINE

"We propose that a 2-month, 10-man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. *The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer."*

L'intelligence artificielle selon John McCarthy

"It is the science and engineering of making intelligent machines, especially intelligent computer programs. It is related to the similar task of using computers to understand human intelligence, but AI does not have to confine itself to methods that are biologically observable."

— John McCarthy

Le Test de Turing

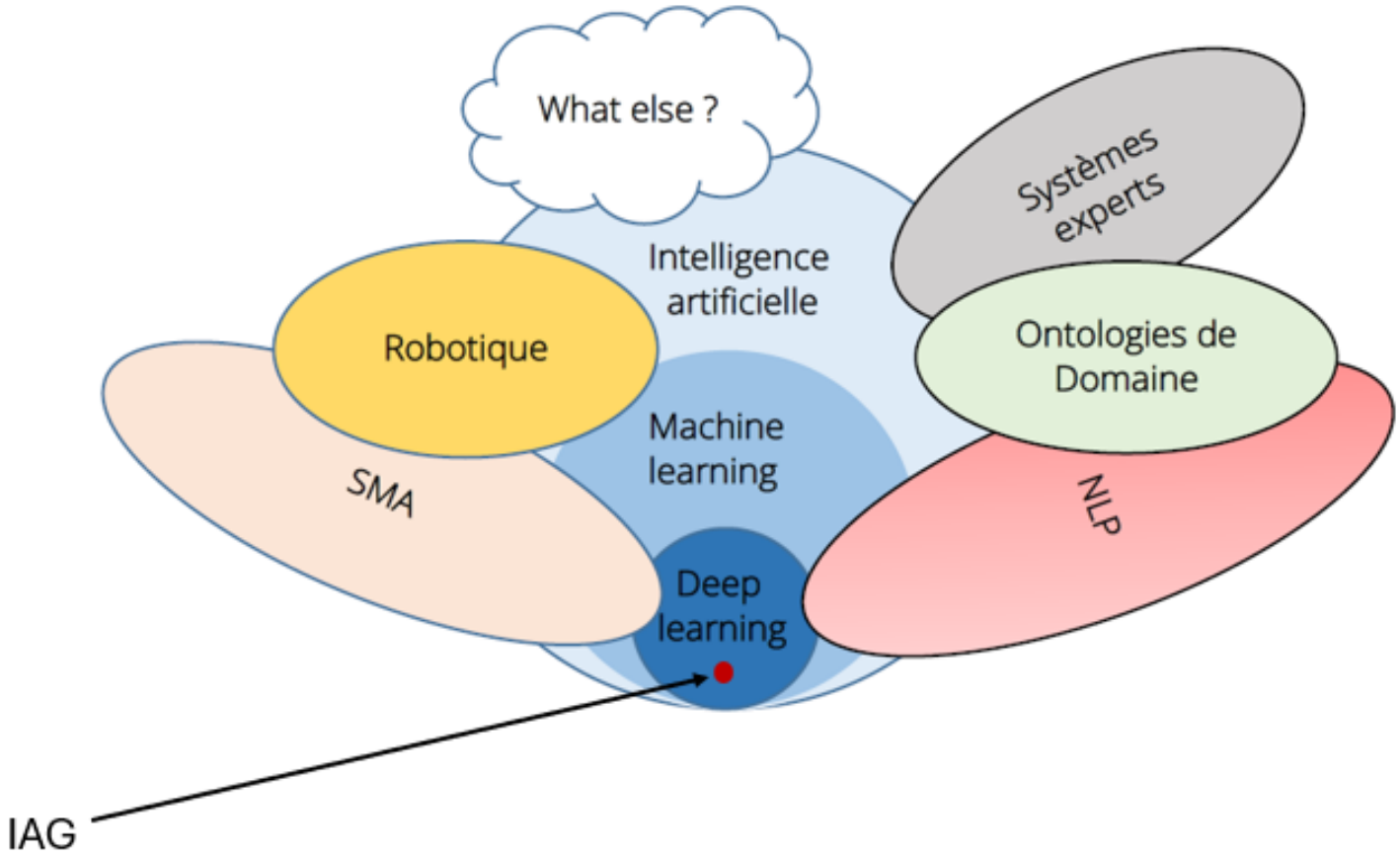
Le Test de Turing, développé par Alan Turing en 1950, est une tentative de mesurer l'intelligence d'une machine, plus précisément de la factulté d'une machine à penser. Cette dernière n'étant pas si évidente à mesurer, le test substitue finalement à la faculté de penser celle de traiter le langage naturel comme un humain.

Les points clés du Test de Turing sont :

- Un interrogateur humain engage une conversation avec un humain et une machine, chacun étant caché de la vue de l'interrogateur.
- Si l'interrogateur ne peut pas déterminer systématiquement quelle est la machine, celle-ci est considérée comme ayant passé le test.
- Le test ne mesure pas la connaissance ou la capacité à être véridique, mais plutôt la capacité de reproduire le comportement humain.

Définition pragmatique de l'intelligence artificielle

Il s'agit d'un ensemble de techniques qui permettent à la machine d'accomplir des tâches qui requièrent traditionnellement une intelligence humaine.



IA forte vs IA faible

La distinction entre IA forte et IA faible se réfère à deux approches conceptuelles différentes dans le domaine de l'intelligence artificielle.

IA faible :

- Aussi connue sous le nom d'IA "étroite", elle est conçue pour effectuer des tâches spécifiques et ne possède pas de conscience.
- Les systèmes d'IA faible agissent et réagissent uniquement en fonction des instructions programmées et des algorithmes spécifiques.
- Exemples : assistants virtuels, systèmes de recommandation, reconnaissance vocale.

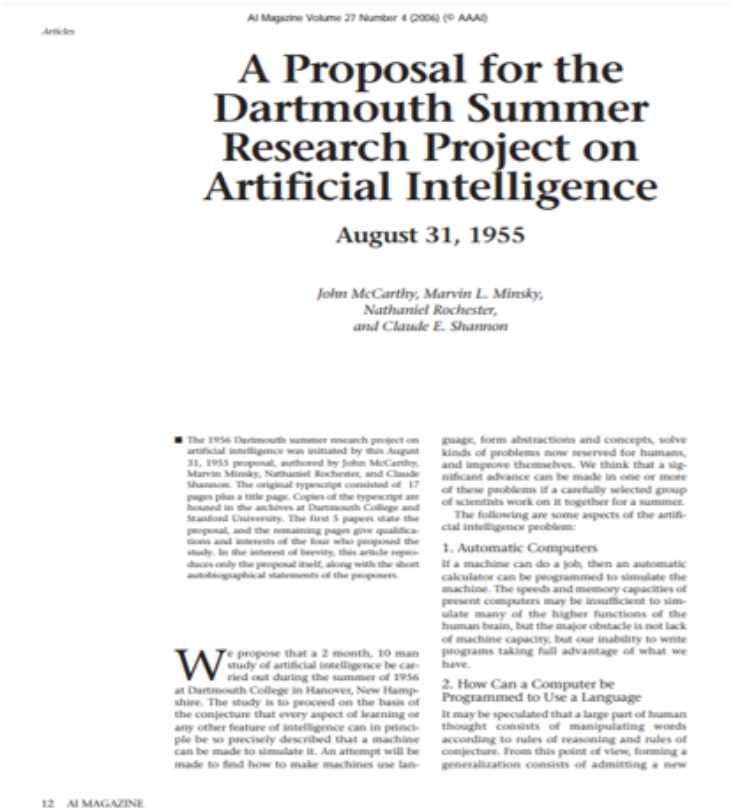
IA forte :

- Vise à créer des machines dotées de capacité de généralisation à l'image de l'intelligence humaine.
- L'IA forte serait capable d'apprendre, de raisonner, de résoudre des problèmes et de prendre des décisions indépendamment.
- À ce jour, l'IA forte reste un objectif à atteindre, qui fait l'objet de recherches intensives.

Introduction au NLP

1956 : conférence de Dartmouth

L'histoire du NLP est intimement liée à celle de l'intelligence artificielle. Il figure même dans le programme de la conférence de Dartmouth, qui a fondé l'IA.



AI Magazine Volume 27 Number 4 (2006) (© AAAI)

Articles

A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence

August 31, 1955

John McCarthy, Marvin L. Minsky, Nathaniel Rochester, and Claude E. Shannon

■ The 1956 Dartmouth summer research project on artificial intelligence was initiated by this August 31, 1955 proposal, authored by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon. The original typescript consisted of 17 pages plus a title page. Copies of the typescript are housed in the archives at Dartmouth College and Stanford University. The first 5 papers state the proposal, and the remaining pages give qualifications and interests of the four who proposed the study. In the interest of brevity, this article reproduces only the proposal itself, along with the short autobiographical statements of the proposers.

We propose that a 2 month, 10 man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer.

The following are some aspects of the artificial intelligence problem:

1. Automatic Computers

If a machine can do a job, then an automatic calculator can be programmed to simulate the machine. The speeds and memory capacities of present computers may be insufficient to simulate many of the higher functions of the human brain, but the major obstacle is not lack of machine capacity, but our inability to write programs taking full advantage of what we have.

2. How Can a Computer be Programmed to Use a Language

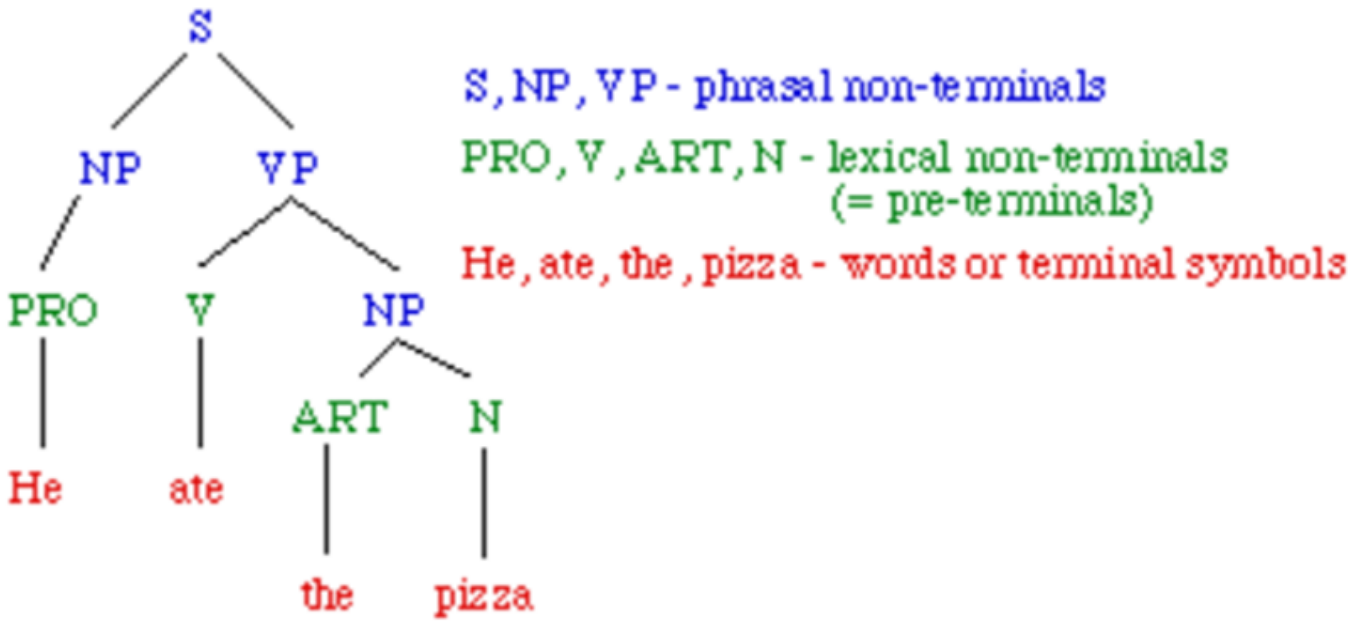
It may be speculated that a large part of human thought consists of manipulating words according to rules of reasoning and rules of conjecture. From this point of view, forming a generalization consists of admitting a new

12 AI MAGAZINE

"We propose that a 2-month, 10-man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. *The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer."*

1970: approches traditionnelles du NLP

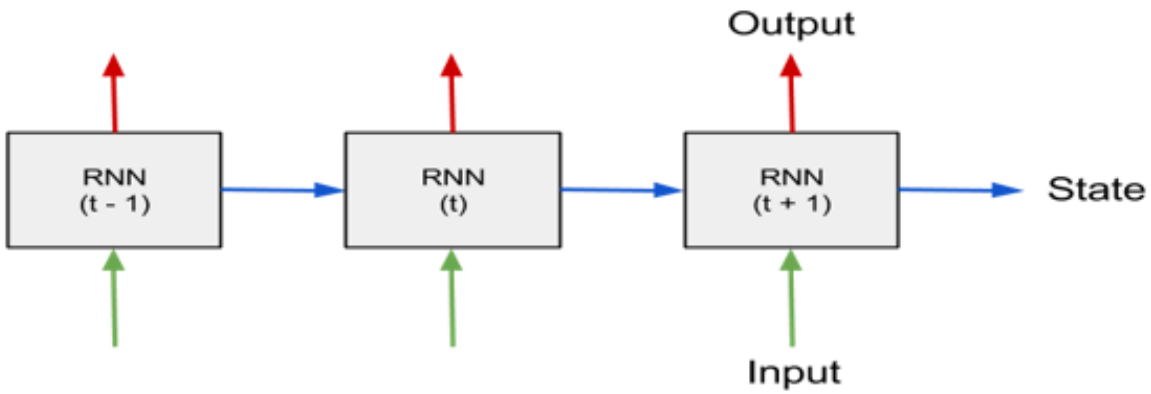
Les modèles traditionnels reposent majoritairement sur une approche linguistique. Ils consistent généralement à parser le langage naturel, même quand celui-ci possède une structure complexe.



1986: Les réseaux de neurons récurrents

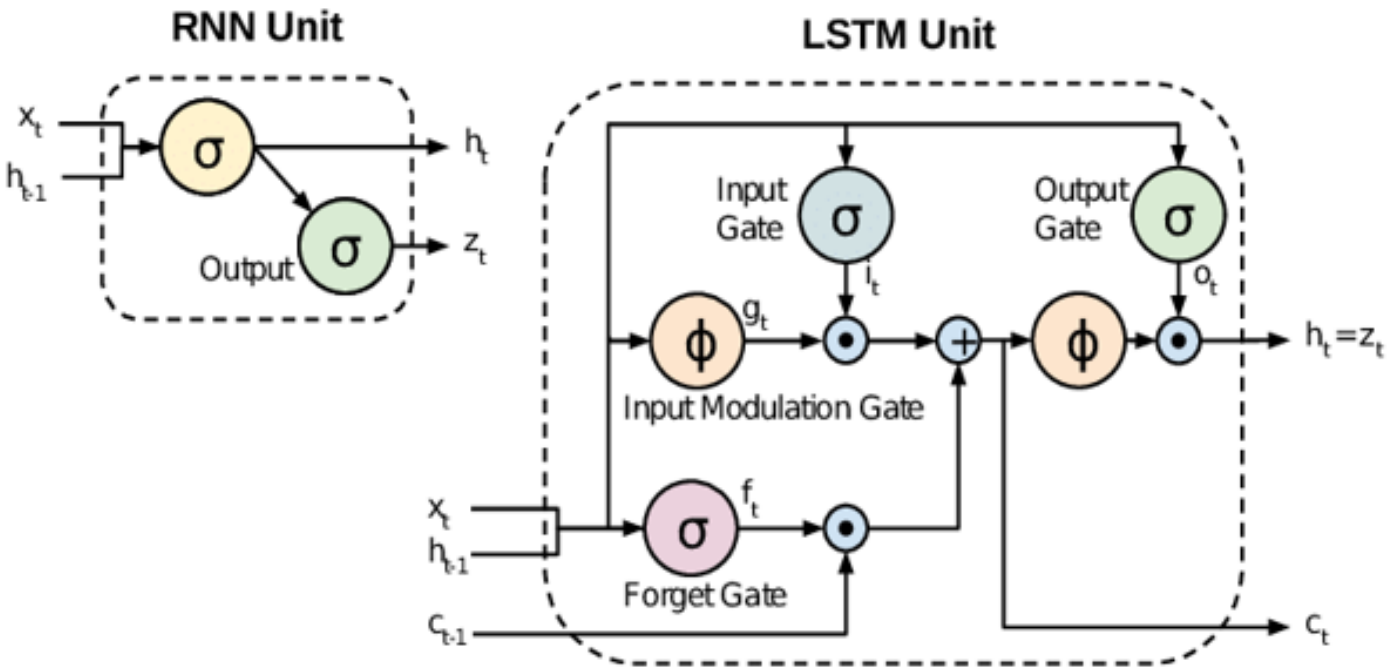
Les RNNs permettent de traiter des séquences ; ils sont ainsi naturellement adaptés au langage naturel. Ils ont cependant rapidement montré des limites.

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure[1].



1997: Long Short-Term Memory

Les LSTMs sont nés pour dépasser les limites des RNNS, en particulier le problème de l'annulation du gradient.



2010s: Renaissance du deep learning

Plusieurs avancées ont été réalisées durant la dernière décennie en *deep learning*, notamment depuis 2012 (AlexNet).

- 2013 : représentations distribuées et Word2Vec
- 2014 : Émergence du mécanisme de self-attention
- 2017 : Émergence des modèles Transformers
- 2018 : ELMo, BERT, GPT.
- 2020 : GPT-3.
- 2022 : ChatGPT, etc.

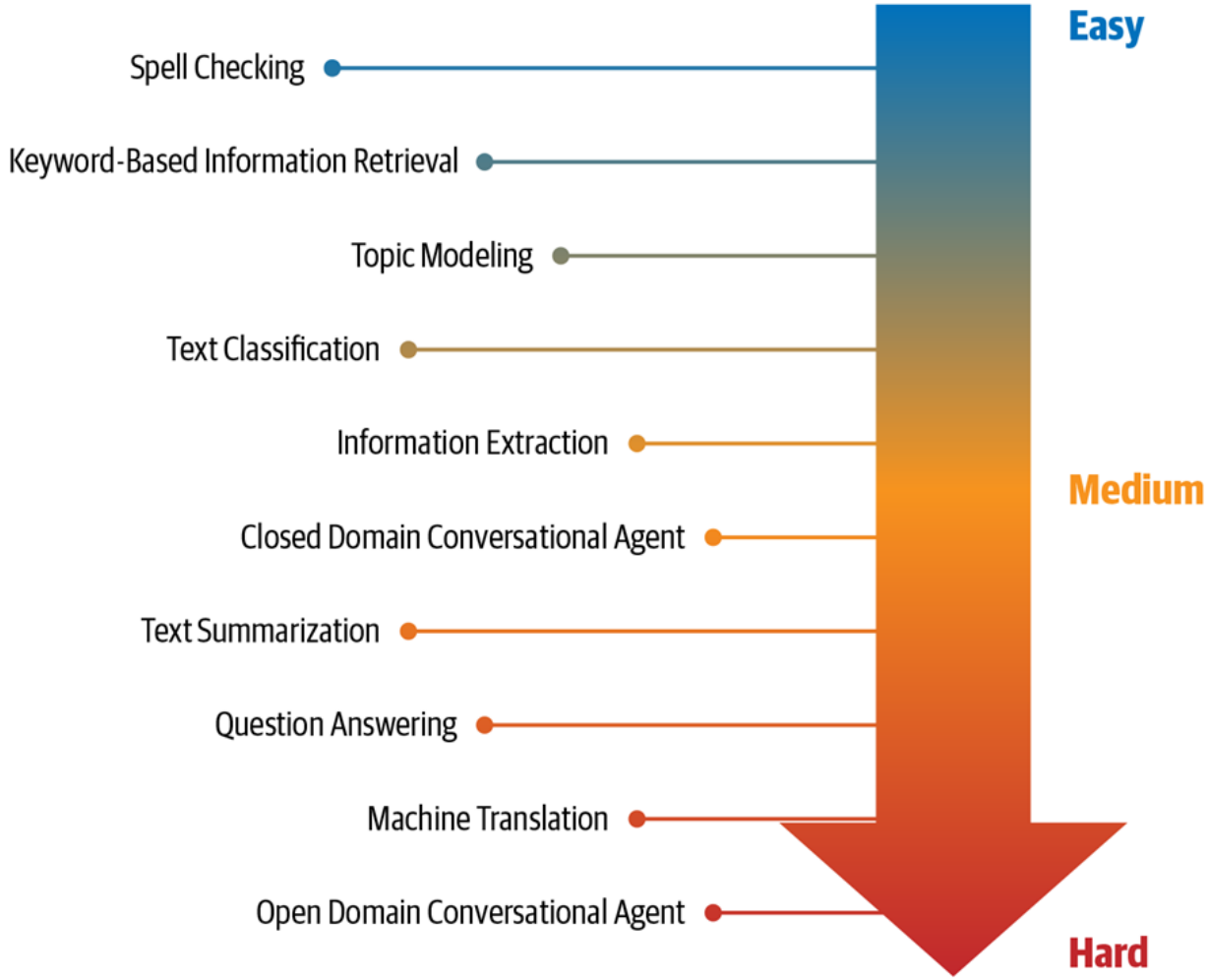
Tâches classiques en NLP

On retrouve dans le NLP un certain nombre de tâches classiques, qui servent à la fois dans les applications pratiques que dans l'évaluation de l'état de l'art.

- Modélisation du langage (*language modeling*)
- Classification de texte
- Extraction d'informations
- Détection de thématiques (*topic modeling*)
- Résumé de texte
- Question/réponse
- Traduction automatique
- Agent conversationnel (domaine ouvert ou fermé)

Classement des tâches en NLP par ordre de difficulté

Le classement ci-dessous est à titre indicatif et sujet à des changements. Certaines tâches, comme la traduction, paraissent aujourd'hui moins difficiles qu'il y a quelques années.



Pourquoi le NLP est si difficile ?

- Le langage naturel est **ambigu**
  - *"Acheter un avocat"*
- Le langage naturel repose sur le **sens commun**
  - Il fait froid *en hiver*
  - Le beurre fond dans le four
- Le langage naturel n'est pas basé sur des **règles**
  - *En bleu adorable fleurit*
    Le toit de métal du clocher. Alentour
    Plane un cri d'hirondelles, autour
    S'étend le bleu le plus touchant. Le soleil
    Friedrich Hölderlin

Pré-traitement des données textuelles

Les données textuelles sont non structurées. Les principales étapes de pré-traitement impliquent :

1. Segmentation des phrases et **tokenisation** ;
2. Suppression de **stop words**, de la ponctuation, etc.
3. **Stemming**, lemmatisation, conversion des majuscules, etc.

Et

- Détection du langage, POS tagging, etc.

Tokenisation

La tokenisation consiste à segmenter une phrase en unités plus petites (*tokens*), qui sont généralement des mots, mais qui peuvent également être des caractères ou ensembles de caractères.

Exemple:

- "Il fait beau aujourd'hui à Paris" $\rightarrow$ ["il", "fait", "beau", "aujourd'hui", "à", "Paris"]
- "Paris" $\rightarrow$ ["P", "a", "r", "i", "s"]

La tokenisation par subword est utilisée d'une manière assez fréquente en **deep learning**.

Stemming and lemmatisation

- Le **stemming** est l'opération qui consiste à supprimer les suffixes et réduire les mots à leur forme de base.
  - marcher → march
  - marchait → march
  - marcha → march
  - marchassiez → march
  - marché → march

- La **lemmatisation** est l'opération qui consiste à réduire un mot à sa racine.
  - nous → nous
  - sommes → être
  - venus → venir
  - faire → faire
  - les → le
  - marchés → marché

Vectorisation

- **One-Hot encoding**: Étant donné un vocabulaire V, chaque mot est représenté par un vecteur binaire (à 0 ou 1) de dimension V.
  - $V = \{Tom, mange, pomme, chien, ciel\}$, où dim$(V) = 5$
  - Tom $=$ [1, 0, 0, 0, 0]
  - mange $=$ [0, 1, 0, 0, 0]
  - pomme $=$ [0, 0, 1, 0, 0]
  - etc.

- **N-grams**
  - "machine learning est très utilisé en NLP."
  - 1-gram: {machine, learning, est, très, utilisé, en, NLP}
  - 2-gram: {machine learning, learning est, est utilisé, utilisé en, en NLP}

Extraction d'informations

L'information peut concerner différentes entités (événements clés, personnes, lieux, etc.). L'extraction d'information inclut :

- Détection d'entités nommées (NER).
- Extraction de mots clés (KPE).
- Extraction de relations.

Entités nommées (NER)

L'extraction d'entités nommées consiste à détecter des informations clés qui sont des noms propres (noms de personnes, de lieux, etc.)

Example:

Established in 1896, the site was taken over by **Askham Bryan** in 2011 and it has 536 students, including apprentices. A group set up to keep it open had tried to find another college to take it over, after two previous bids were deemed unsuitable. **Tim Whitaker**, chief executive officer and principal at **Askham Bryan College**, said he regretted "the upset" the closure and job losses will cause. "Whilst it was very disappointing that the strategic review didn't receive a sustainable option for **Newton Rigg** campus, we welcome the plans for the preservation of land-based provision in **Cumbria**. "We will support and work with those involved in these plans, to ensure that current students and future applicants interested in land-based courses have a smooth transition." He added that the college had "always been clear" educational provision would not continue at **Newton Rigg** from July 2021, and students, staff and the local community were told in 2020.

Extraction de relations

L'extraction de relations consiste à détecter des relations entre deux entités nommées.

Example:

Established in 1896, the site was taken over by **Askham Bryan** in 2011 and it has 536 students, including apprentices. A group set up to keep it open had tried to find another college to take it over, after two previous bids were deemed unsuitable. **Tim Whitaker**, chief executive officer and principal at **Askham Bryan College**, said he regretted "the upset" the closure and job losses will cause.

"Whilst it was very disappointing that the strategic review didn't receive a sustainable option for **Newton Rigg** campus, we welcome the plans for the preservation of land-based provision in **Cumbria**."

"We will support and work with those involved in these plans, to ensure that current students and future applicants interested in land-based courses have a smooth transition."

He added that the college had "always been clear" educational provision would not continue at **Newton Rigg** from July 2021, and students, staff and the local community were told in 2020

Extraction de mots clés

L'extraction de mots clés consiste à extraire les mots clés les plus importants, permettant de capturer le contenu d'un texte.

Example:

Established in 1896, the site was taken over by **Askham Bryan** in 2011 and it has 536 students, including apprentices. A group set up to keep it open had tried to find another college to take it over, after two previous bids were deemed unsuitable. **Tim Whitaker**, chief executive officer and principal at **Askham Bryan College**, said he regretted "the upset" the closure and job losses will cause.

"Whilst it was very disappointing that the strategic review didn't receive a sustainable option for **Newton Rigg** campus, we welcome the plans for the preservation of land-based provision in **Cumbria**."

"We will support and work with those involved in these plans, to ensure that current students and future applicants interested in land-based courses have a smooth transition."

He added that the college had "always been clear" educational provision would not continue at **Newton Rigg** from July 2021, and students, staff and the local community were told in 2020

Part of Speech tagging

Part of speech consiste à détecter la catégorie grammaticale d'un mot au sein d'un texte.

Exemple :

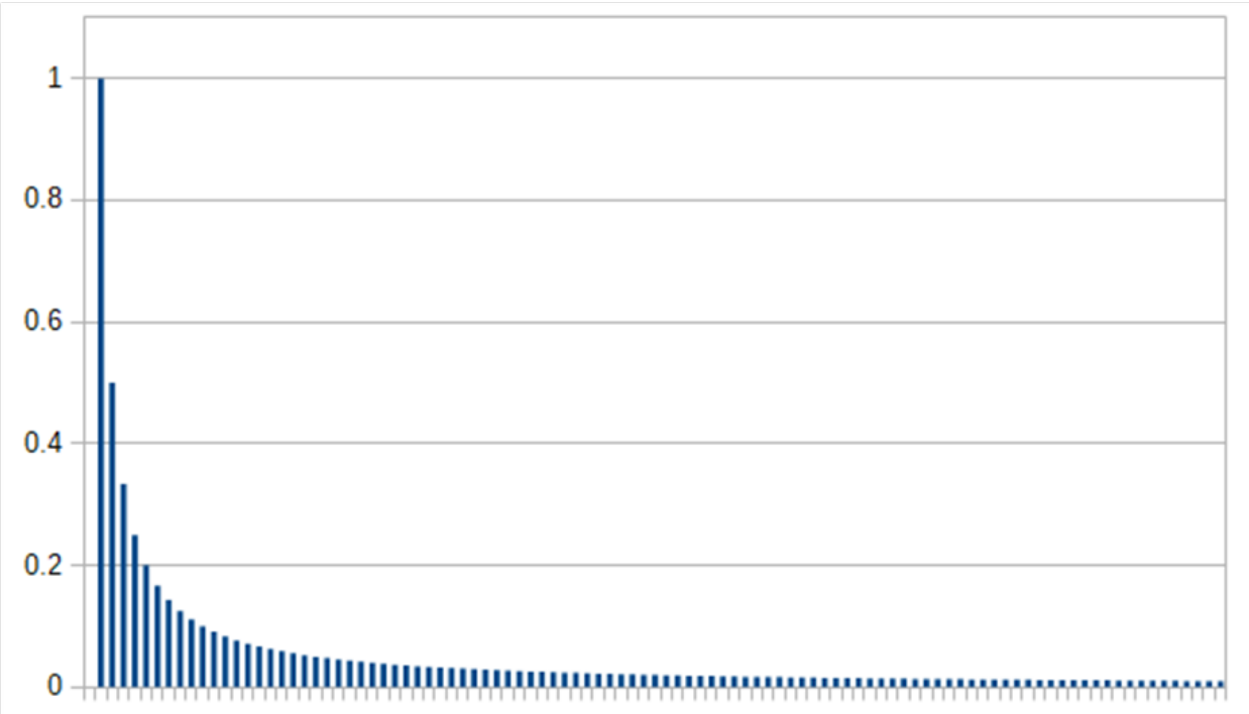
"this is a very simple example"

[('this', 'DT'), ('is', 'VBZ'), ('a', 'DT'), ('very', 'RB'), ('simple', 'JJ'), ('example', 'NN')]

Techniques statistiques pour le NLP

Principe des techniques statistiques pour le NLP

- Un document est caractérisé par la distribution des mots qui le composent.
- La distribution des mots obéit généralement à une loi de puissance (ou de Zipf). La majorité du langage que nous utilisons ordinairement est composé d'un nombre très limité de mots.



Term Frequency

- Term Frequency (TF): la fréquence avec laquelle un terme t apparaît dans un document d.

$$\text{TF}(t, d) = \frac{(\text{Number of occurrences of term } t \text{ in document } d)}{(\text{Total number of terms in the document } d)}$$

Exemple : étant donné 1000 mots.

Terme	**Fréquence**	**TF score**
chat	15	0.015
lait	4	0.004
mange	6	0.006
feu	3	0.003
...	...	...

Inverse Document Frequency

- Inverse Document Frequency (IDF): permet de pénaliser les termes les plus fréquents et de donner plus de poids aux termes les moins fréquents (plus parlants).

$$\text{IDF}(t) = \log_e \left(\frac{\text{Total number of documents in the corpus}}{\text{Number of documents with term } t \text{ in them}} \right)$$

Example: given 100 documents

Terme	**Nombre de documents**	**IDF score**
chat	70	0.36
lait	9	2.41
manger	80	0.22
feu	5	3.00
...	...	...

Term Frequency – Inverse Document Frequency

- TF-IDF score est le produit de Tf et IDF.

$$\text{TF-IDF Score} = \text{TF Score} \times \text{IDF score}$$

Terme	**TF score**	**IDF score**	**TF-IDF score**
chat	0.015	0.36	0.0054
lait	0.004	2.41	0.0096
manger	0.006	0.22	0.0013
feu	0.003	3.00	0.0090
...	...	...	...

Topic Modeling

- Topic modeling is a technique for:
  "automatically organizing, understanding, searching and summarizing large electronic archives." - David Blei

- Découvrir des thématiques sous-jacentes à un document ou un ensemble de documents (Google News, etc.)
- Annoter ou caractériser les documents selon les thématiques.

Machine learning

Qu'est-ce que la machine learning ?

L'apprentissage automatique est une branche de l'intelligence artificielle qui consiste à doter les machines de la capacité d'apprendre à partir de données sans que celles-ci ne soient explicitement programmées pour exécuter des tâches spécifiques.

- **IA discriminative** : Modélise la frontière entre les classes.
  Exemples : régression logistique, SVM, réseaux de neurones pour la classification.
- **IA générative** : modélise la distribution des données pour en générer de nouvelles.
  Exemples : GAN, VAE, modèles de langage comme GPT.
- **Apprentissage par renforcement (Reinforcement Learning)** : apprend à prendre des décisions séquentielles dans un environnement.
  L'agent n'imite pas ou ne classe pas, il agit pour maximiser une récompense.
  Exemples : Q-learning, PPO, RLHF pour le fine-tuning des LLMs.

Machine learning discriminatif

L'apprentissage automatique est une branche de l'intelligence artificielle qui consiste à doter les machines de la capacité d'apprendre à partir de données sans que celles-ci ne soient explicitement programmées pour exécuter des tâches spécifiques.

Le machine learning englobe deux grandes familles d'apprentissage :

- **Supervisé** : Les algorithmes apprennent à partir de données étiquetées pour faire des prédictions ou classifications.

- **Non supervisé** : L'apprentissage est effectué sur des données non étiquetées pour trouver des structures cachées.

L'apprentissage supervisé

L'apprentissage supervisé consiste à apprendre un modèle qui associe une étiquette (*label*) à un ensemble de caractéristiques (*features*).

- **Inputs** : un jeu de données *annotées* pour entraîner le modèle.
  - Exemple : des textes (tweets, etc.) avec les *sentiment* associés, positifs ou négatifs.
- **Output** : une étiquette pour un point de donnée inconnu par le modèle.

L'apprentissage supervisé se décline lui-même en deux grandes familles :

- **La classification** : prédire une catégorie ou une classe.
  - Exemple : prédire l'étiquette d'une image (chat, chien, etc.), le sentiment associé à un texte, le centre d'intérêt d'un client à partir de ses commentaires, etc.
- **La régression** : prédire une valeur continue (un nombre réel typiquement).
  - Exemple : prédire le prix d'un appartement, la lifetime value d'un client, etc.

Classification

Exemple de classification : maintenance prédictive

Âge (ans)	Temp. (°C)	Vib. (mm/s)	H./sem.	Pannes	Entretien	Label (y)
2	55	1.2	40	0	Oui	Pas de panne
6	70	3.8	60	2	Non	Panne
4	65	2.5	50	1	Oui	Pas de panne
8	80	4.1	55	3	Non	Panne
3	58	1.9	45	0	Oui	Pas de panne

Régression

Régression – volume journalier distribué dans un dépôt de carburant

Jour	Temp. (°C)	Prix (€/L)	Promo (0/1)	Stock restant (%)	Volume distribué (m^3/jour)
Lun	23	1,98	0	65	**380**
Mar	25	1,95	1	60	**420**
Mer	22	1,96	0	58	**360**
Jeu	30	1,94	1	55	**500**
Ven	28	1,97	0	52	**470**
Sam	26	1,99	0	50	**430**
Dim	20	2,01	0	70	**280**

L'apprentissage non supervisé

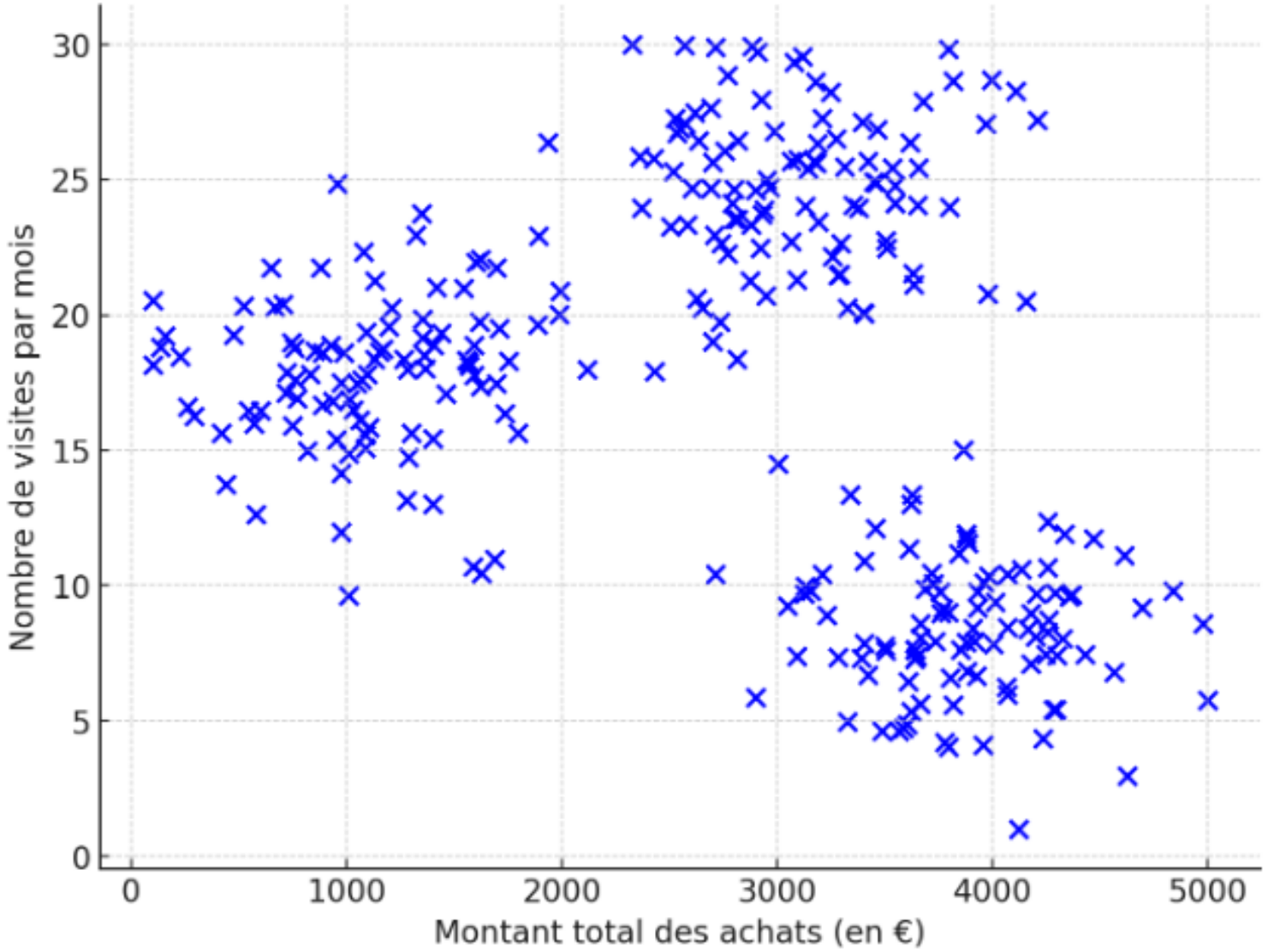
L'apprentissage non supervisé se réfère à l'utilisation de modèles d'apprentissage automatique pour identifier des patterns et des structures dans des données qui ne sont pas étiquetées.

Principales typologies de l'apprentissage non supervisé :

- **Clustering** : Regroupement de points de données similaires ensemble. Exemple : segmentation de marché, regroupement social.
- **Réduction de dimension** : Représenter les données avec moins de variables en conservant l'essentiel de l'information. Exemple : analyse en composantes principales (ACP).

Clustering

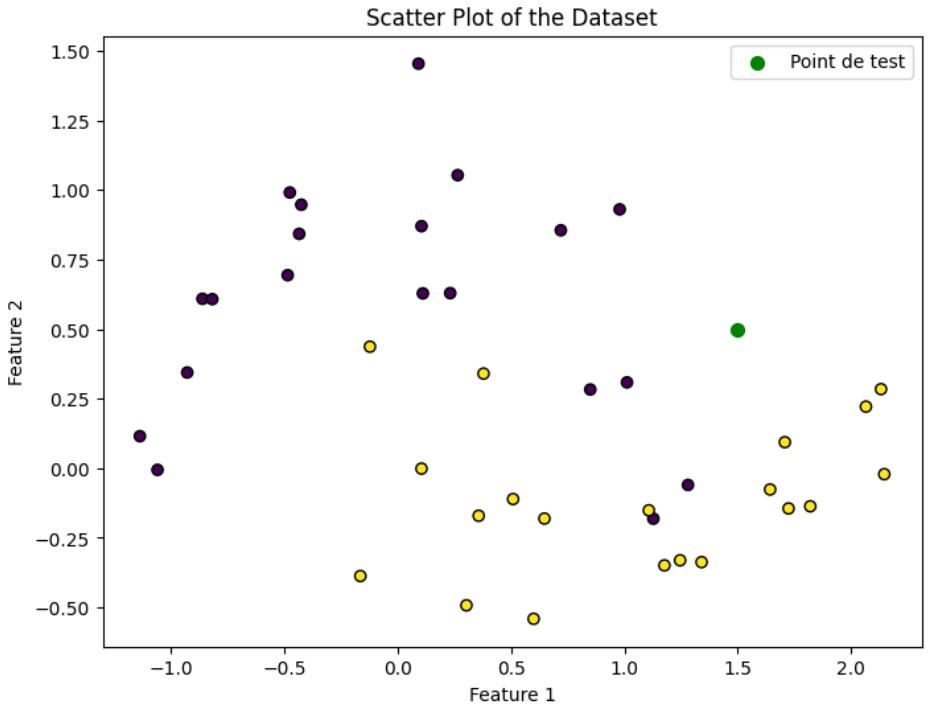
Exemple de clustering : segmentation clients



Apprentissage supervisé

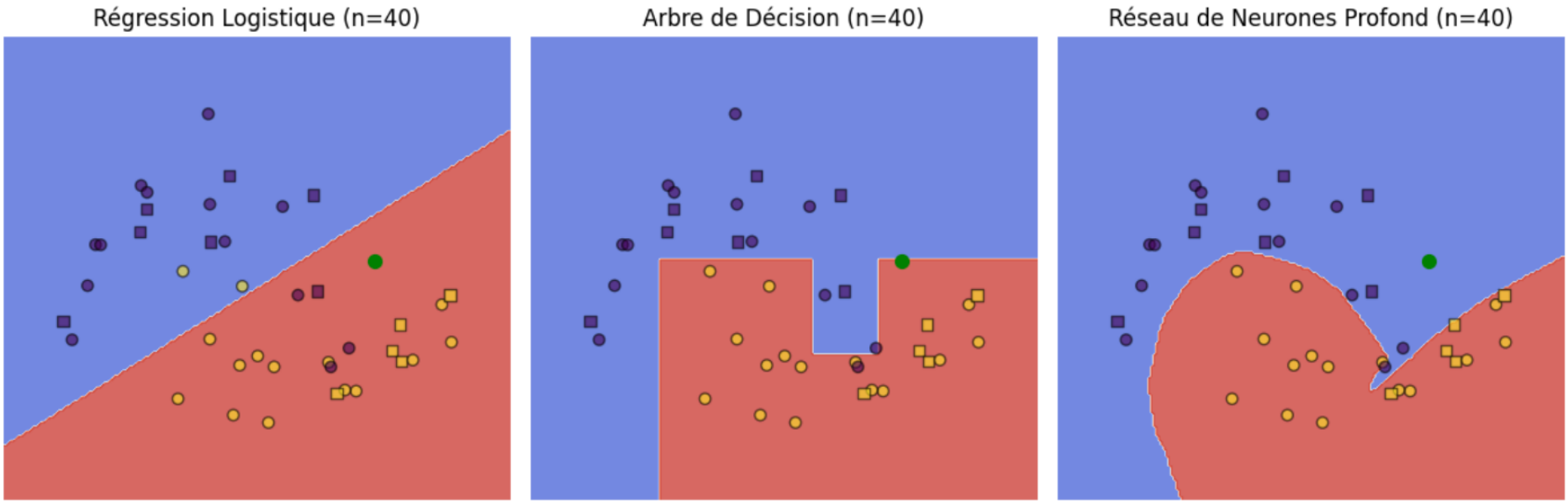
L'apprentissage supervisé comme un problème d'induction

- **Définition** : L'apprentissage supervisé consiste à apprendre une fonction f qui mappe les entrées X aux sorties y, à partir d'un ensemble d'exemples d'entraînement (X, y).
- **Induction** : Le modèle induit une règle générale à partir de données particulières, dans le but de généraliser à de nouvelles instances.
- **Problème de généralisation** : Comment garantir que le modèle apprend une règle qui s'applique à de nouvelles données ?

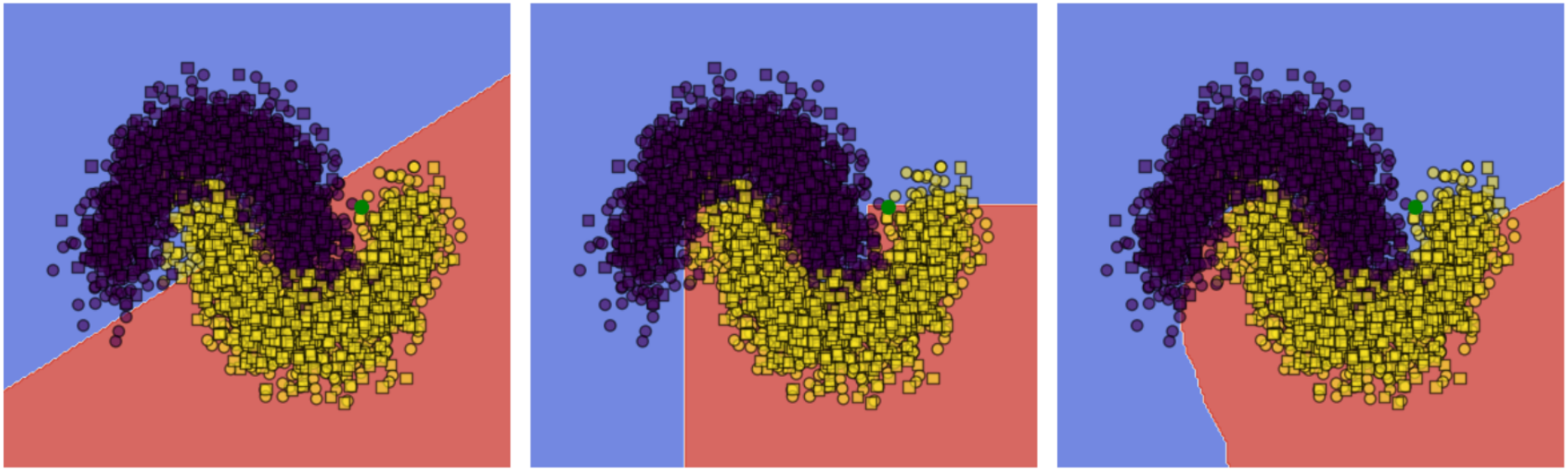


Une indétermination intrinsèque pour le choix du modèle

Il y a une infinité de manières d'induire un modèle à partir d'un échantillon de données d'entraînement.



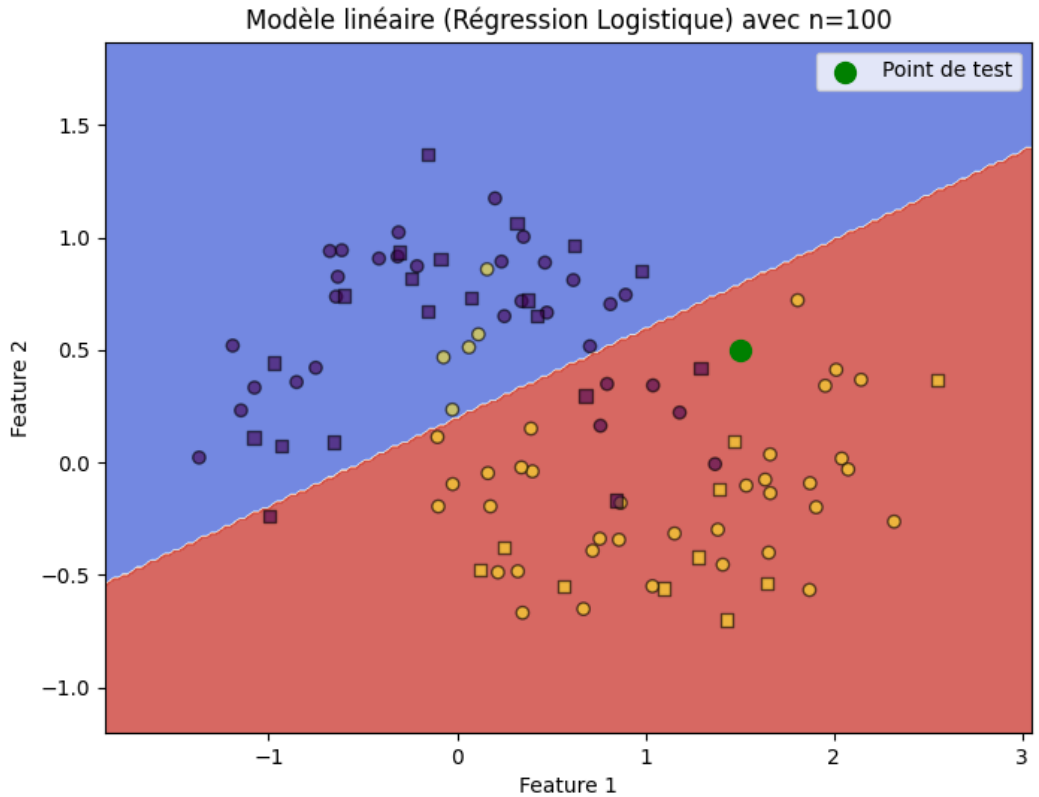
Sous-apprentissage et surapprentissage sur les données d'entraînement



Sous-apprentissage

Definition

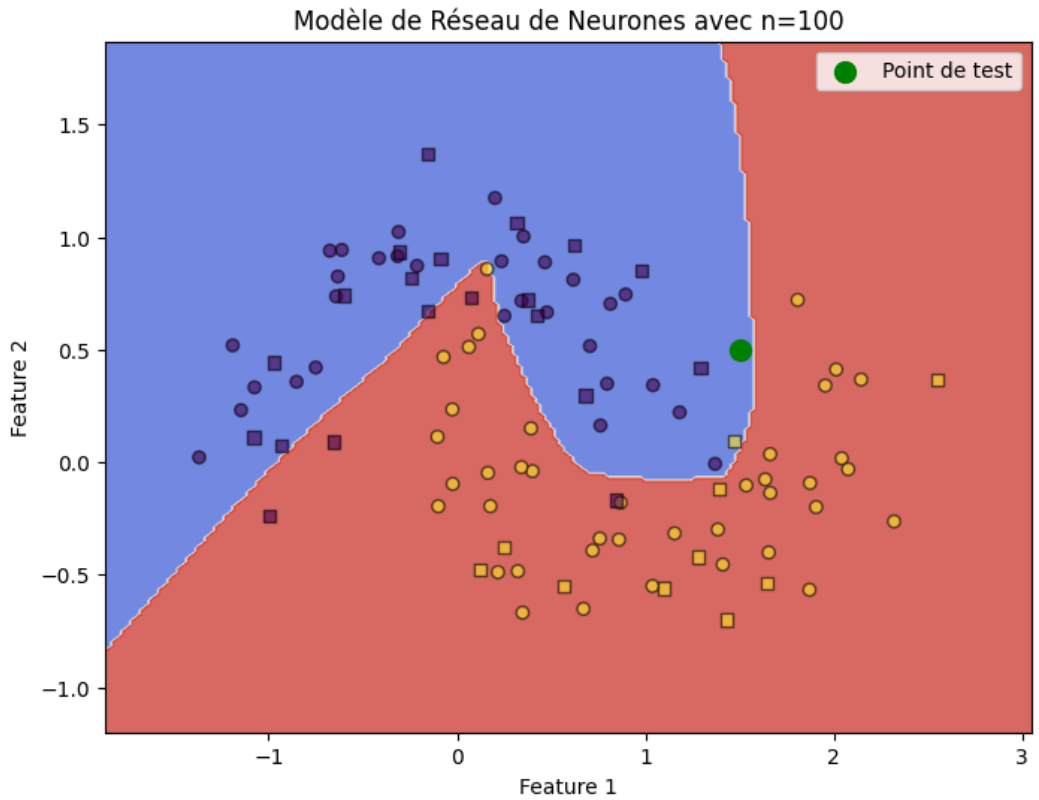
On dit qu'un modèle de machine learning est en régime de sous-apprentissage (underfitting) lorsqu'il n'arrive pas à capturer la complexité (l'information) présente dans le jeu de données d'entraînement.



Sur-apprentissage

Definition

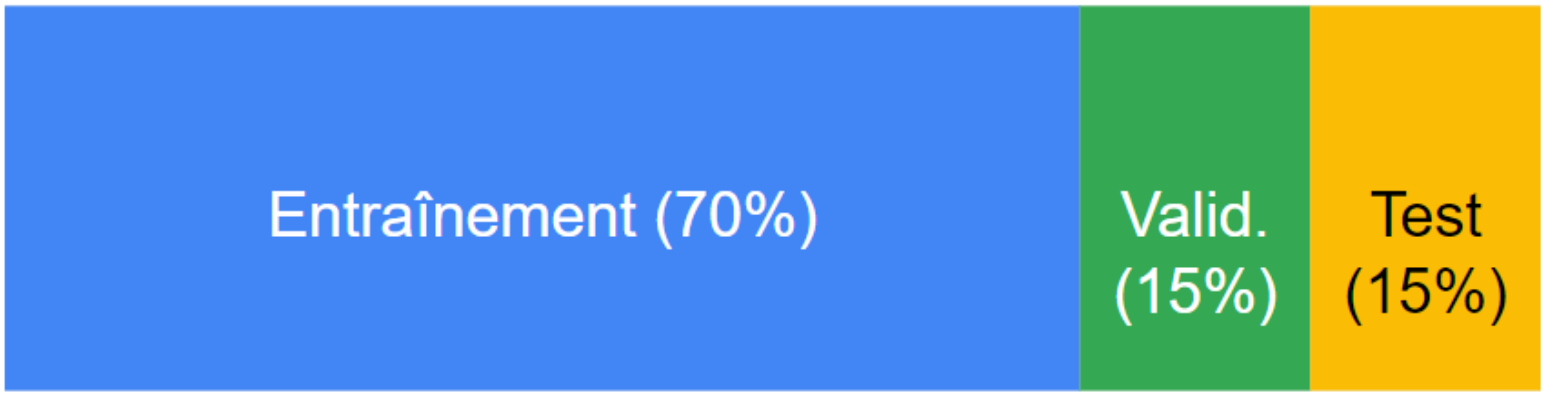
On dit qu'un modèle de machine learning est en régime de sur-apprentissage (overfitting) lorsqu'il n'arrive pas à généraliser à des données non encore observées, i.e. lorsqu'il est trop adapté aux données d'entraînement.



Sélection de modèle

Pour sélectionner le modèle le plus pertinent par rapport à une métrique donnée, on applique la méthodologie suivante :

- On partitionne le jeu de données disponible en trois parties : un jeu d'entraînement, un jeu de validation et un jeu de test.
- On entraîne M modèles sur le jeu d'entraînement.
- On évalue les performances respectives des M modèles sur le jeu de validation et on sélectionne le meilleur.
- Le modèle sélectionné est ensuite évalué sur le jeu de test. Idéalement, le jeu de test est ainsi utilisé une seule fois.



Exemple : sélection de modèle

Modèle	Précision Entraînement	Précision Validation
Régression Logistique	0.90	0.88
Arbre de Décision	0.95	0.92
Réseau de Neurones Profond	1	0.90

Table: Comparaison des précisions des modèles sur les jeux d'entraînement et de validation

Métriques de performance : régression

On dispose d'un certain nombre de métriques pour évaluer les performances des modèles de machine learning. Celles-ci peuvent être divisées en deux catégories.

Régression

- L'erreur quadratique moyenne (MSE) : elle est définie comme la moyenne des carrés des écarts entre les prédictions et les valeurs observées.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(f(x_i) - y_i)^2$$

- La racine carrée de l'erreur quadratique moyenne (RMSE) :

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(f(x_i) - y_i)^2}$$

Métriques de performance : classification 1/2

Accuracy : L'accuracy est la métrique de base qui permet d'évaluer les performance d'un modèle de classification. Elle est définie comme :

$$\text{Accuracy} = \frac{\text{Nombre de prédictions correctes}}{\text{Nombre total de prédictions}}$$

Matrice de confusion : La matrice de confusion est une représentation permettant d'offrir plus de finesse par rapport à l'accuracy, notamment quand le jeu de données est déséquilibré (présence de classes majoritaires). Elle compare les prédictions du modèle avec les valeurs réelles et est structurée comme suit :

		Valeur Prédite	
		Positif	Négatif
Valeur Réelle	Positif	Vrai Positif (VP)	Faux Négatif (FN)
	Négatif	Faux Positif (FP)	Vrai Négatif (VN)

Métriques de performance : classification 2/2

A partir de la matrice de confusion, on peut dériver d'autres métriques :

- Précision : elle est définie comme la proportion des prédictions correctes parmi toutes les prédictions positives :

$$\text{Précision} = \frac{VP}{VP + FP}$$

- Rappel (recall) : il représente la proportion des vrais positifs correctement prédits par le modèle.

$$\text{Rappel} = \frac{VP}{VP + FN}$$

- Score F1 (F1-score) : Le score F1 est défini comme la moyenne harmonique de la précision et du rappel.

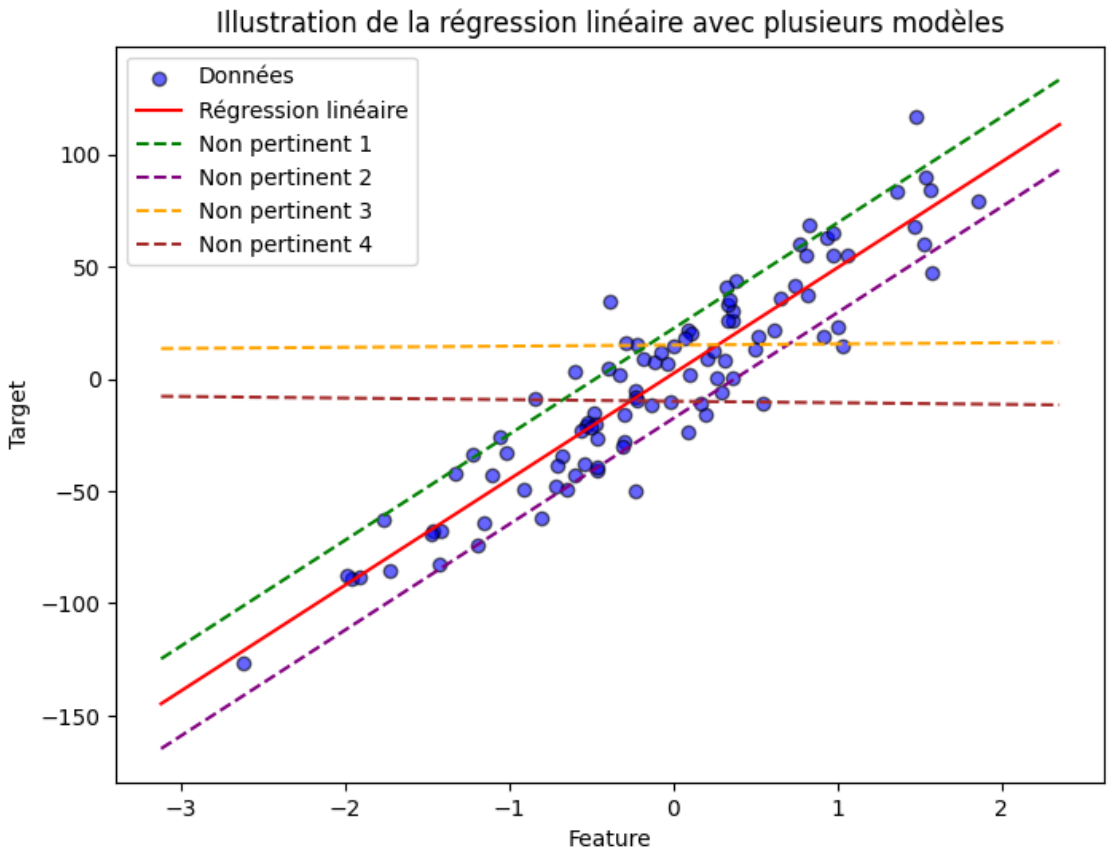
$$\text{Score F1} = 2\frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

Modèles classiques de machine learning

Régression linéaire simple

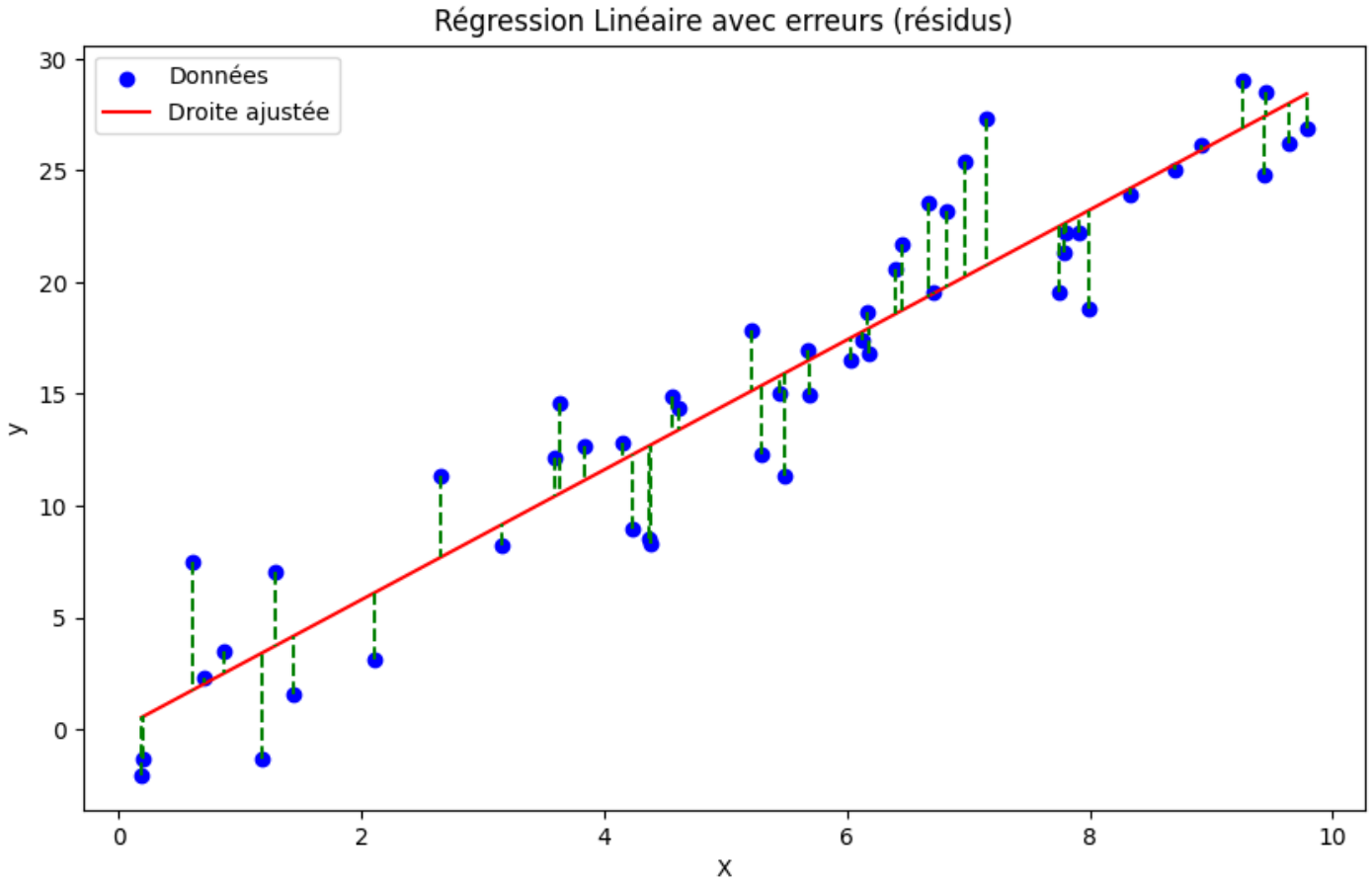
Soit un ensemble de n obeservations $x_1, x_2, ..., x_n$ avec les labels correspondants $y_1, y_2, ..., y_n$, on cherche le modèle linéaire qui ajuste le mieux ces données.

$$\hat{y} = \beta_0 + \beta_1 x$$



Minimisation du risque empirique

L'erreur de prédiction pour la i ième observation est : $e_i = y_i - \hat{y}_i$. où $\hat{y}_i = \beta_0 + \beta_1 x_i$.



Exemple : prédire le prix d'un logement en fonction de la surface 1/2

Soit un dataset de 100 points où x représente la surface (en m²) et y le prix des logements (en €). Les 4 premières lignes sont :

x (Surface m²)	y (Prix en €)
40	120 000
65	200 000
80	250 000
55	160 000
⋮	⋮

Les coefficients β_0 et β_1 sont calculés à partir des formules suivantes :

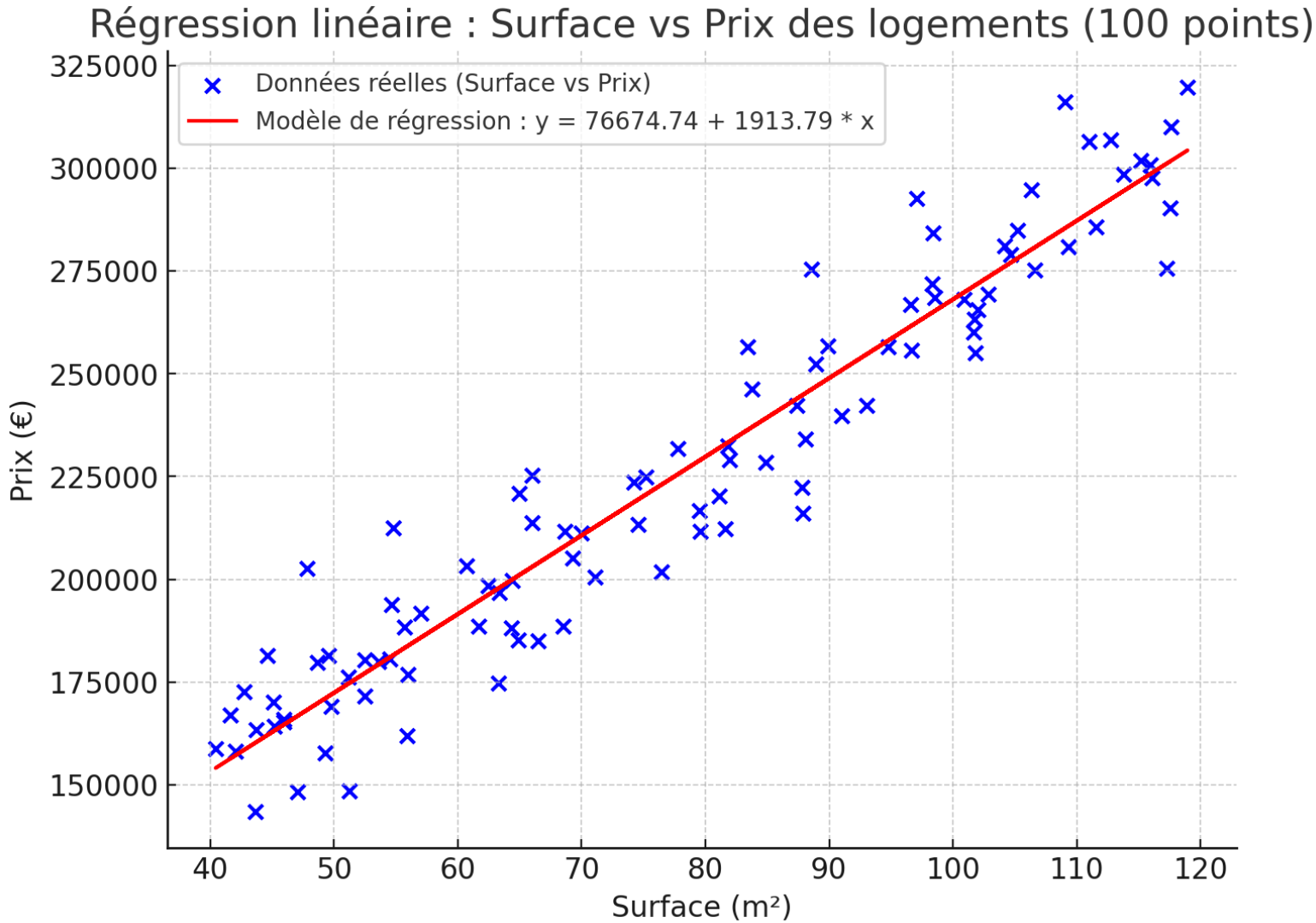
$$\beta_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2} = \frac{1234567}{56789} = 2173.15$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x} = 200000 - 2173.15 \times 60 = 69410.5$$

L'équation de la droite de régression obtenue est donc :

$$y = 69410.5 + 2173.15 \cdot x$$

Exemple : prédire le prix d'un logement en fonction de la surface 2/2



Régression linéaire multiple

On considère n observations $X^1, X^2, ..., X^n$ où chaque observation X^i est désormais un vecteur ayant p composantes (p variables explicatives).

$$X^i = \begin{pmatrix} x_1^i \\ x_2^i \\ \vdots \\ x_p^i \end{pmatrix}$$

La régression linéaire s'écrit alors :

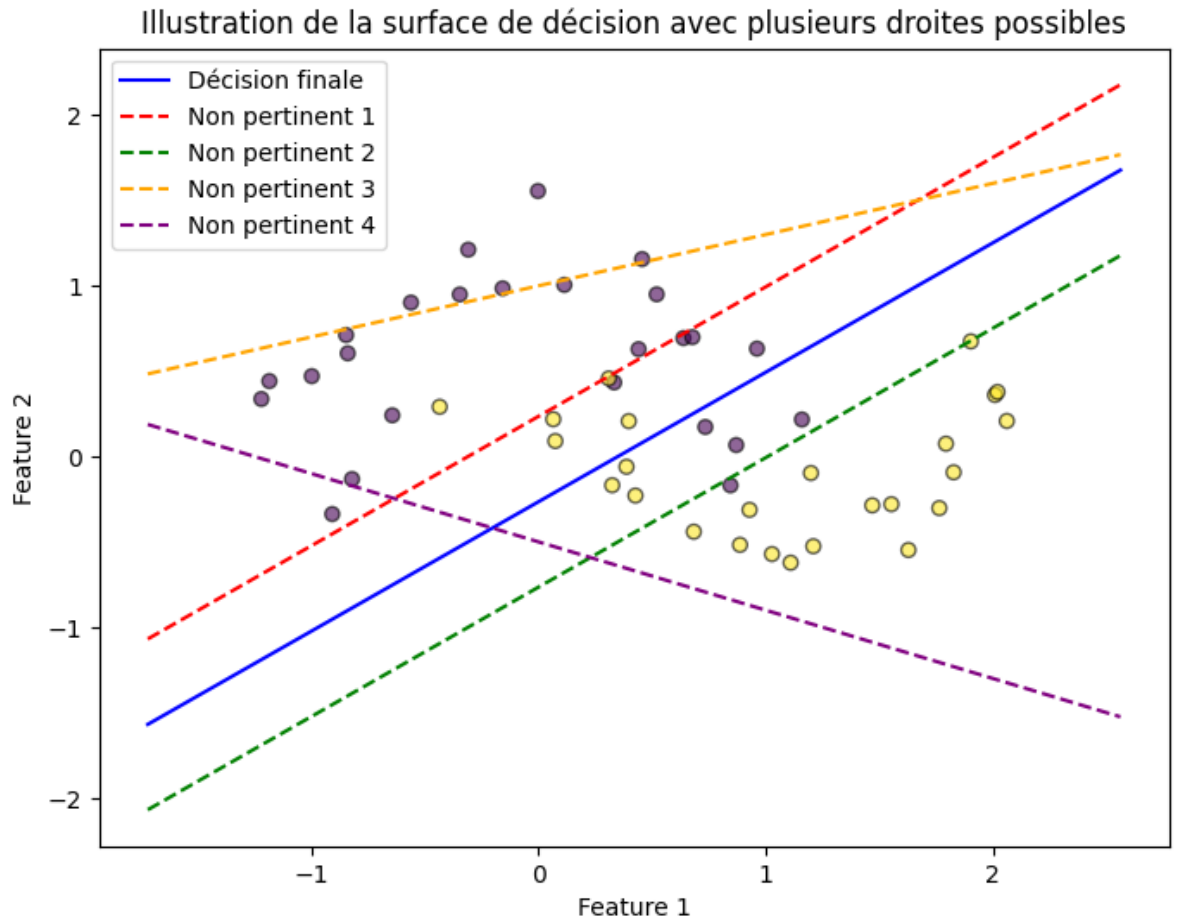
$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p$$

Les coefficiens $\beta_0, \beta_1, \ldots, \beta_p$ sont déterminés par la méthode des moindres carrés :

$$\underset{\beta_0,\beta_1,\ldots,\beta_p}{\arg\min} \sum_{i=1}^{n} \left(y^i - (\beta_0 + \sum_{j=1}^{p} \beta_j x_j^i) \right)^2$$

Principe de la régression logistique

- **Définition** : La régression logistique est un algorithme de classification linéaire utilisé pour prédire la probabilité qu'une observation appartienne à une classe donnée.



Régression logistique : introduction

La régression logistique est une technique d'analyse statistique utilisée pour modéliser la probabilité d'une variable dépendante binaire. C'est un cas particulier de modèle linéaire généralisé qui est utilisé pour des problèmes de classification.

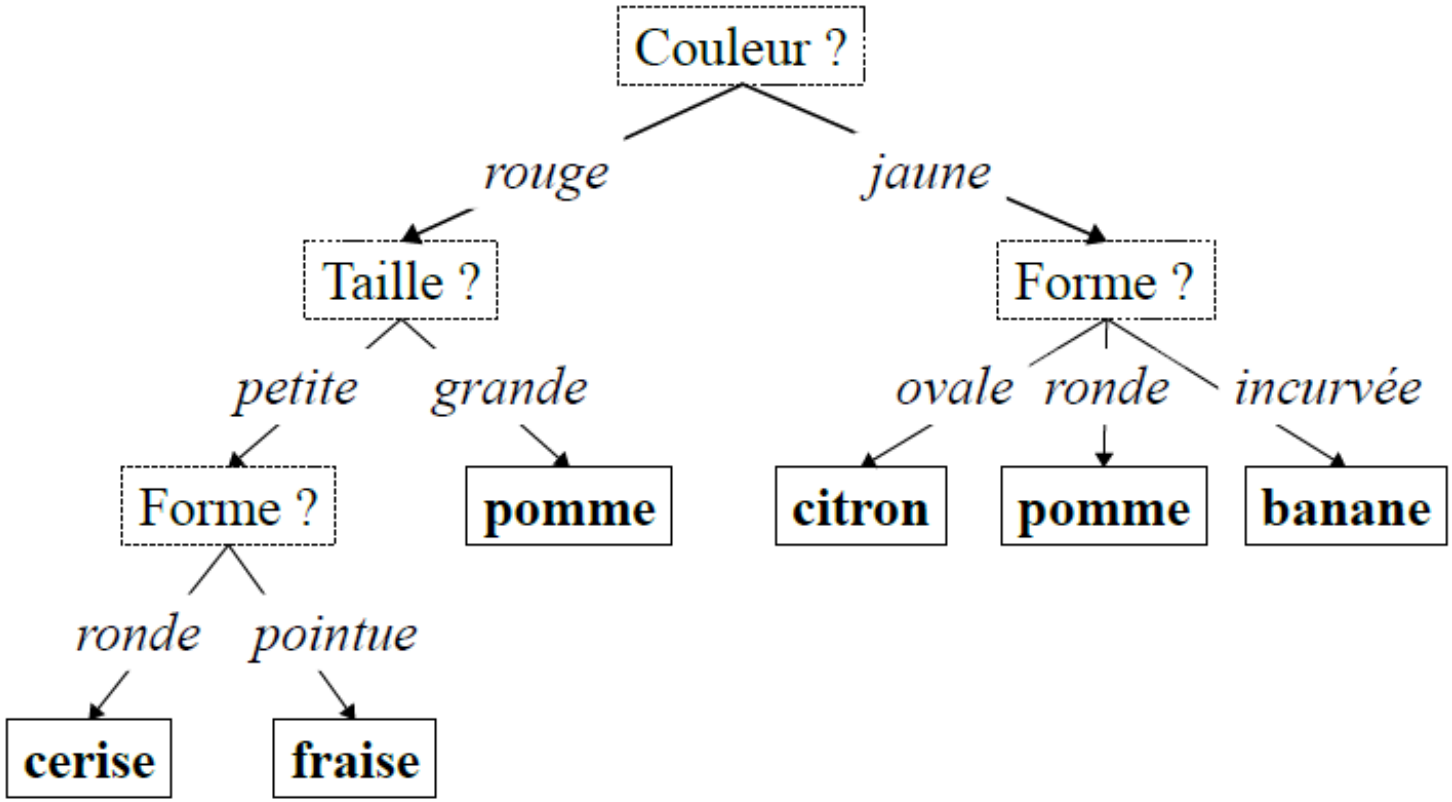
Principes de la régression logistique :

- **Variable dépendante** : On cherche la probabilité que la variable dépendante (y) appartienne à une classe (0 ou 1, vrai ou faux, succès ou échec). Autrement dit, on cherche à modéliser $P(y = 1)$ en fonction des variables dépendantes (explicatives) x.
- **Odds ratio** : Plus concrètement, on cherche à exprimer la côte anglaise (odd ratio) en fonction des variables dépendantes (x).

$$\ln \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_p x_p$$

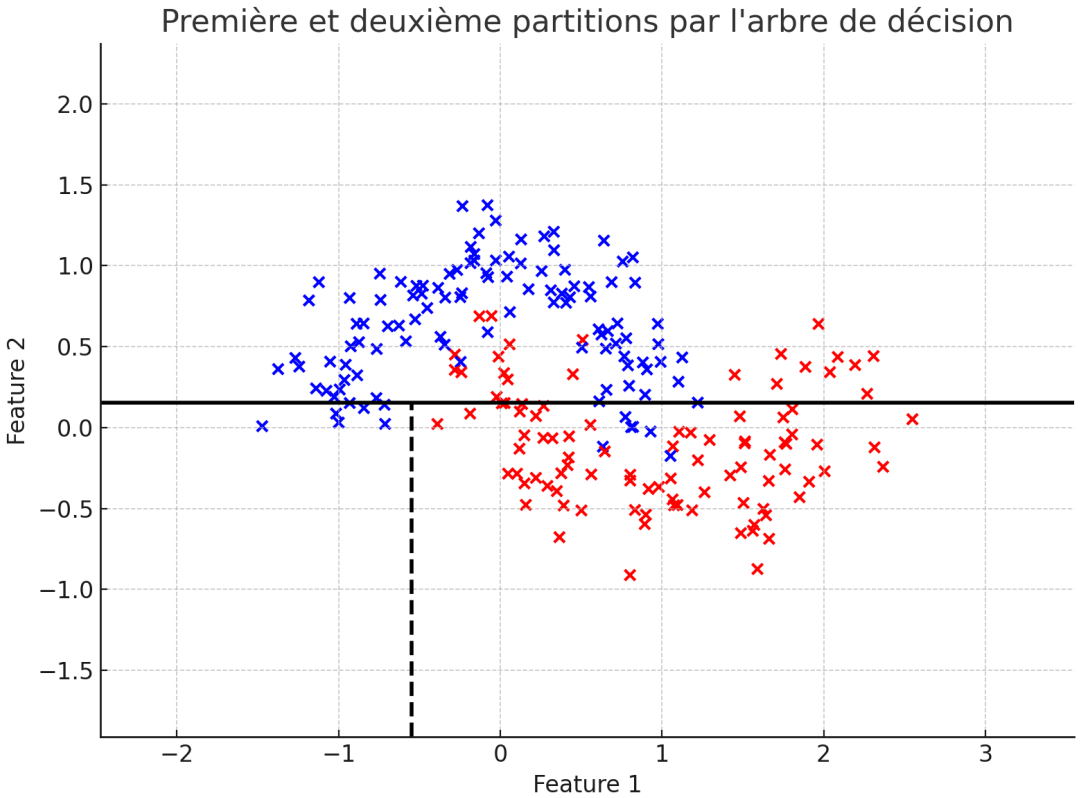
Arbres de décision

Les arbres de décision sont des modèles dont le processus de décision est hiérarchique et prend la forme d'un arbre.



Entraînement des arbres de décision

Les arbres de décision sont généralement entraînés à l'aide de la technique CART (Classification And Regression Trees).
Etant donné un ensemble d'observations $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$, les arbres de décision partitionnent cet espace en plusieurs régions $R_1, R_2, \ldots, R_m$.



Critères d'optimisation

Le critère d'optimisation dépend de la tâche en question.

- **Classification**
  - Indice de Gini simplifié : $\sum_{k=1}^{K} p_{mk}(1 - p_{mk})$
  - Entropie croisée : $-\sum_{k=1}^{K} p_{mk} \ln(p_{mk})$

- **Régression**

$$\sum_{i=1}^{n} (y_i - f(x_i))^2$$

Forêts aléatoires (random forests)

La technique des forêts aléatoires (random forests) consiste à appliquer une approche de type bagging sur les arbres de décision.

L'algorithme random forests suit cette procédure :

- Tirer par bootstrap B échantillons de taille n à partir de l'ensemble D.
- Pour chaque échantillon tiré, construire un arbre en répétant les étapes suivantes jusqu'à atteindre $n_{\min}$.
  - Tirer d'une manière aléatoire m variables parmi les p variables.
  - Sélectionner la meilleure variable avec le meilleur point de partitionnement.
  - Partitionner le nœud en deux sous-branches.
- Agréger les arbres construits.

  Les prédictions sont agrégées selon qu'il s'agisse de régression ou de classification :

- Régression : moyenne $f^B(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$
- Classification : vote majoritaire.

Forêts aléatoires : remarques

- L'évaluation *out of bag* (OOB) utilise, pour chaque observation, les arbres dont l'échantillon bootstrap ne contient pas cette observation. Elle permet d'estimer les performances de généralisation.
- Le nombre de variables tirées pour chaque noeud est généralement donné par $\sqrt{p}$ pour la classification et $\frac{p}{3}$ pour la régression. Cet hyperparamètre dépend cependant du problème considéré.
- Lorsque le nombre de variables est élevé alors que le nombre de variables réellement pertinentes est faible, la probabilité que les m variables sélectionnées pour chaque partitionnement incluent des variables pertinentes devient faible, et les performances du modèle en termes de généralisation peuvent se détériorer considérablement.
- L'algorithme random forests permet de restituer des informations sur l'importance des variables (feature importance).

Apprentissage non supervisé

Apprentissage non supervisé

Dans l'apprentissage non supervisé, on considère n observations sans labels. On s'intéresse fondamentalement à la probabilité jointe de ces observations.

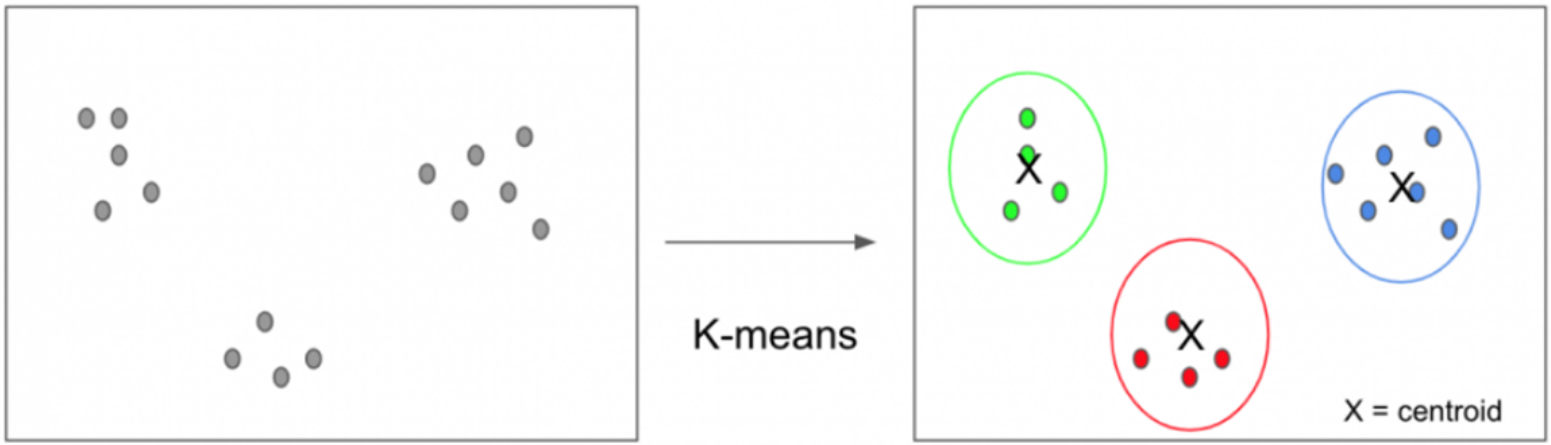
On peut distinguer deux grandes catégories d'apprentissage non supervisé :

- **Clustering (partionnement)** : cela consiste à patitionner les n observations en K groupes pertinents (généralement le critère de pertinence a une signification d'un point de vue métier).

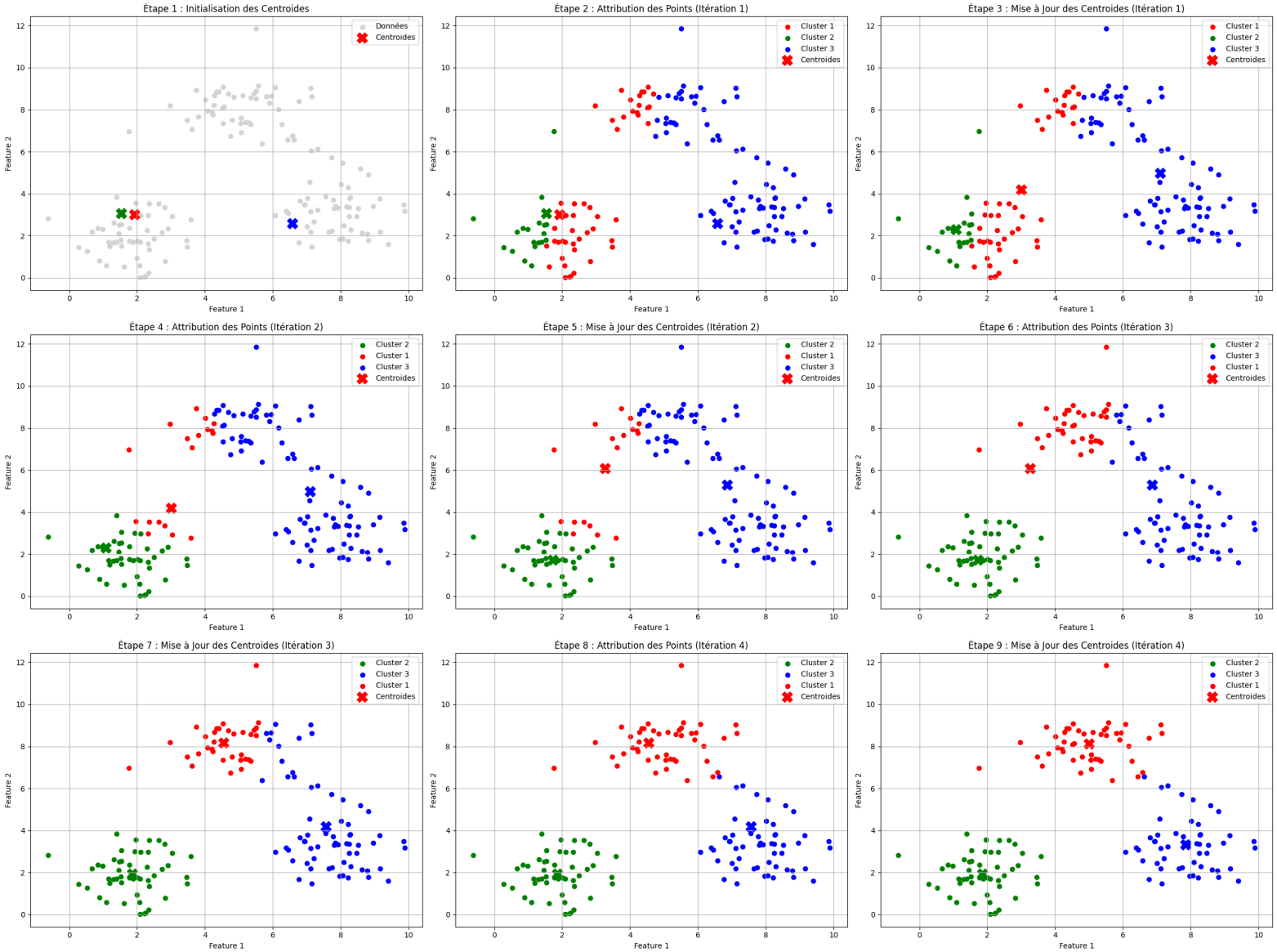
- **Réduction de dimension** : il s'agit de projeter les observations dans un espace de dimension plus faible. L'ACP construit des axes qui conservent le maximum de variance des données.

K-means

L'algorithme de Lloyd tente de regrouper les données en clusters en minimisant les distances entre les points d'un même cluster tout en maximisant les distances entre points appartenant à différents clusters.



Algorithme de Lloyd

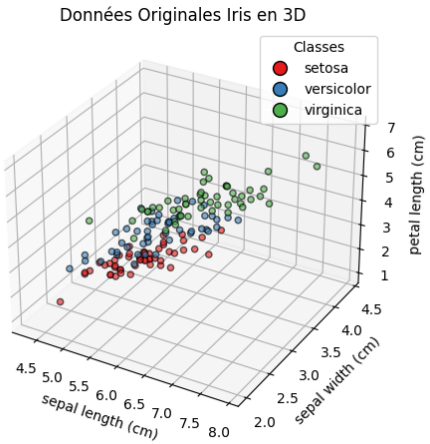


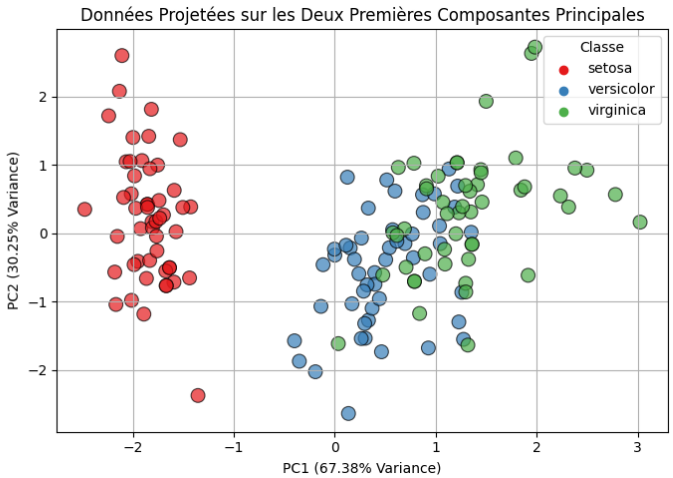
Remarques

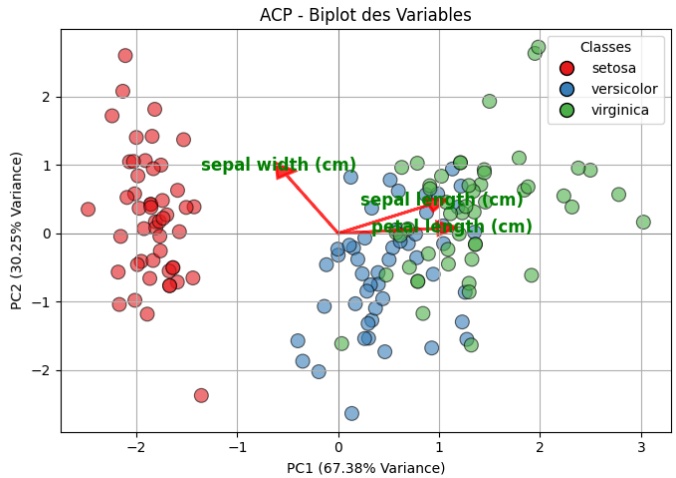
- L'algorithme des k-means étant basé sur une distance euclidienne, il est nécessaire de normaliser les données avant de l'exécuter.
- L'algorithme des k-means est très sensible aux données aberrantes (outliers). Il faut donc considérer les données d'une manière attentive.
- Les centroïdes étant initialisés d'une manière aléatoire, les clusters obtenus ne sont pas stables ; les clusters peuvent changer d'une exécution à l'autre. Il existe cependant une variante plus stable, appelée k-means++, qui permet de sélectionner les centroïdes d'une manière semi-aléatoire.
- Il est possible de partitionner les données avec une métrique plus générale que la distance euclidienne. On peut définir un algorithme k-means à noyau sur un espace de Hilbert pour aller au-delà de la métrique euclidienne.
- **K-means n'est pas adapté aux données en grande dimension**.

Analyse en Composantes Principales (ACP)

L'Analyse en Composantes Principales (ACP) est une technique statistique de réduction de dimensionnalité. Elle transforme les données en un nouveau système de coordonnées où la plus grande variance est capturée sur les premiers axes, appelés composantes principales.







Formulation de l'ACP

Soit X une matrice de données de dimension $n \times p$ (n observations, p variables), centrée (moyenne nulle). L'ACP cherche à trouver les vecteurs propres et les valeurs propres de la matrice de covariance $C = \frac{1}{n-1} X^T X$.

La matrice de covariance C peut être décomposée comme suit :

$$C = VLV^T$$

où $V = [u_1, u_2, \ldots, u_p]$ est la matrice des vecteurs propres et L est une matrice diagonale des valeurs propres λ_k.

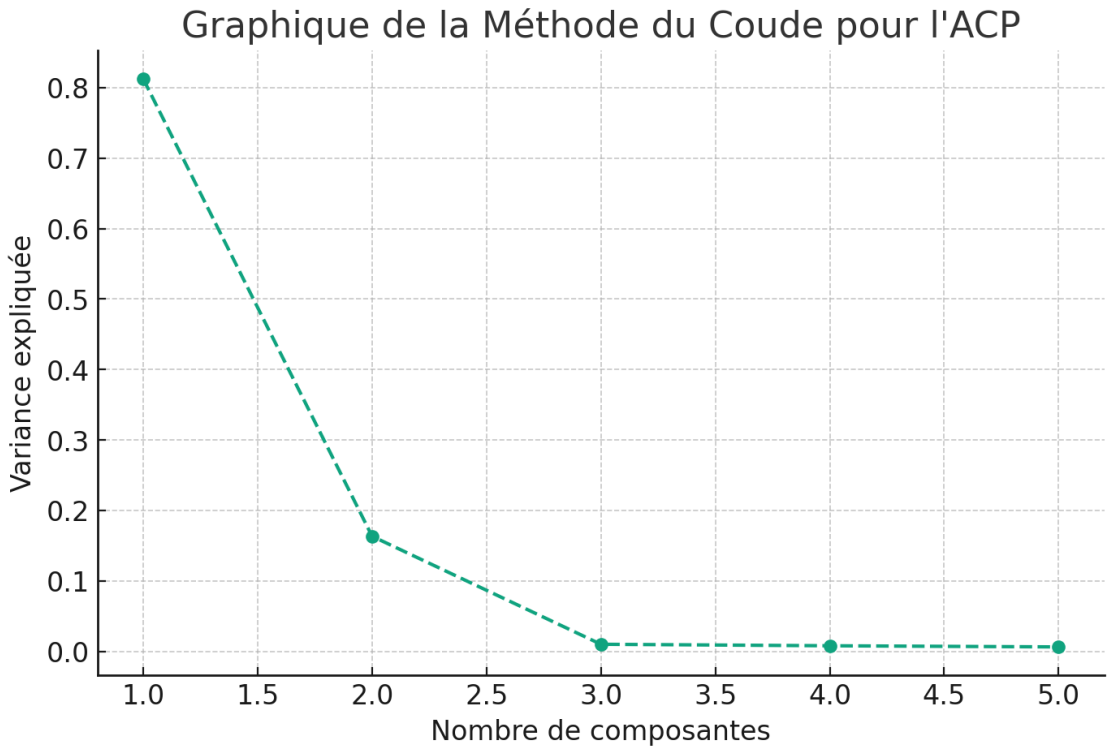
La contribution de chaque composante principale à la variance totale est donnée par :

$$\frac{\lambda_k}{\sum_{i=1}^{p} \lambda_i}$$

Choix du nombre de composantes principales

Le nombre de composantes à retenir est déterminé en fonction du pourcentage de variance totale que l'on souhaite expliquer.

On utilise généralement la méthode du coude (Scree plot). Il s'agit d'un graphique montrant la proportion de la variance expliquée en fonction du nombre de composantes.



Introduction au deep learning

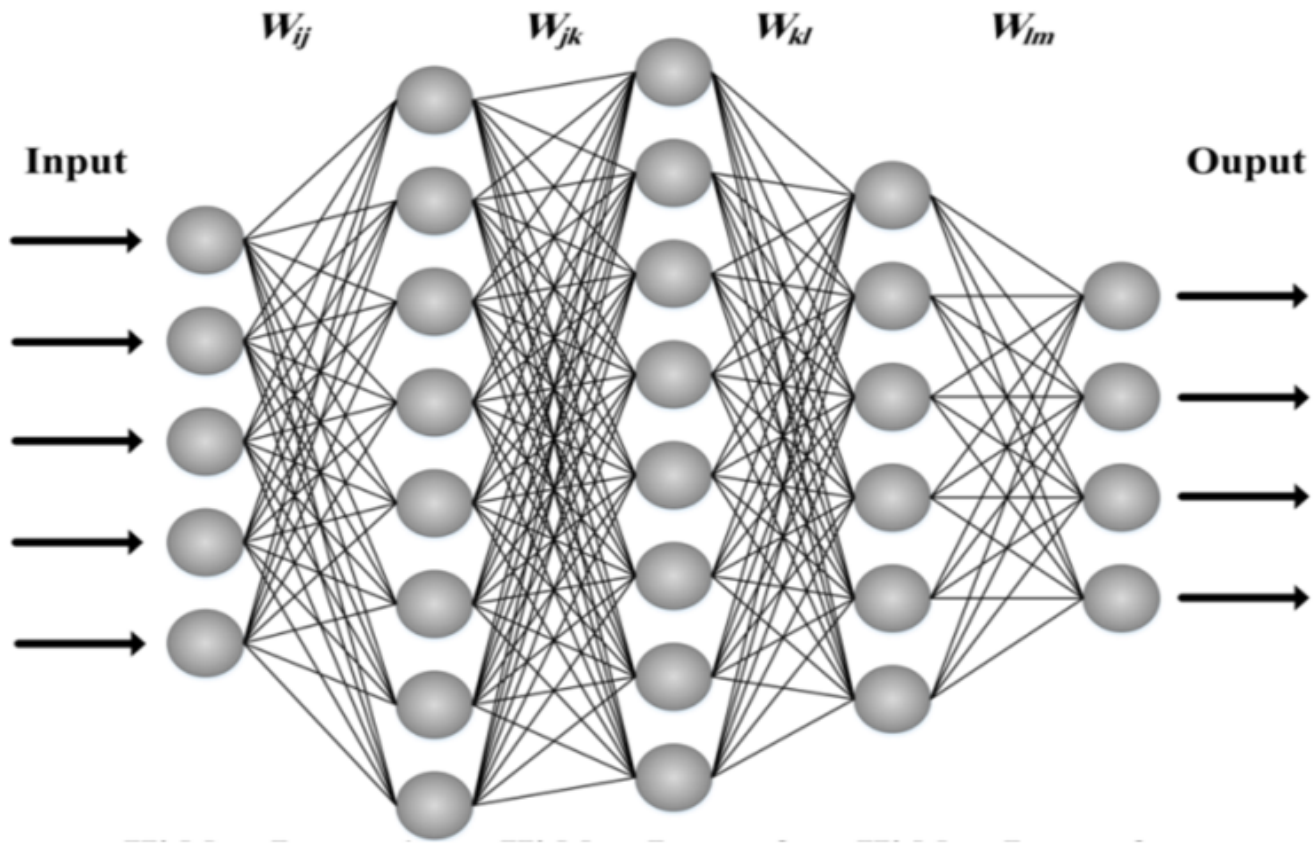
Introduction aux réseaux de neurones

Définition : Les réseaux de neurones sont des modèles computationnels inspirés par le fonctionnement des neurones dans le cerveau humain. Ils sont capables d'apprendre des tâches complexes en modélisant des relations non linéaires entre les entrées et les sorties.

Caractéristiques :

- **Extraction automatique des features** : Capacité d'adaptation et d'extraction des features à partir des données sans programmation explicite.
- **Modélisation non linéaire** : Aptitude à capturer des relations complexes dans les données.
- **Flexibilité** : Applicables à un large éventail de tâches et de typologies de données (images, langage naturel, données graphiques, etc.).

Illustration d'un réseau de neurones classique



Le neurone artificiel

Modèle Mathématique : Chaque neurone artificiel effectue une somme pondérée de ses entrées, ajoute un biais, et passe le résultat à travers une fonction d'activation pour obtenir la sortie.

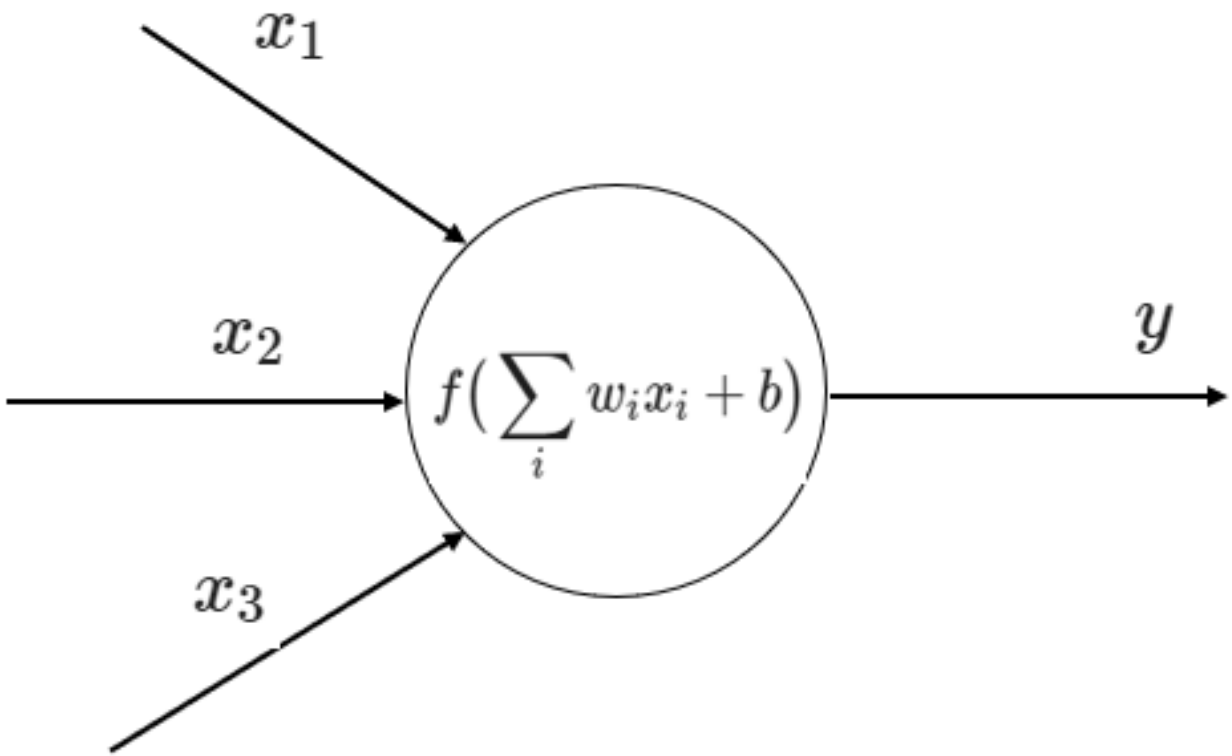
$$z_i = \sum_{j=1}^{m} w_{ij} x_j + b_i$$

$$a_i = f(z_i)$$

où :

- x_j représente l'entrée du neurone,
- w_{ij} est le poids associé à l'entrée x_j,
- b_i est le biais du neurone,
- f est la fonction d'activation,
- z_i est le terme de préactivation (somme pondérée avant activation),
- a_i est la sortie activée du neurone.

Illustration d'un neurone artificiel



Fonctions d'activation

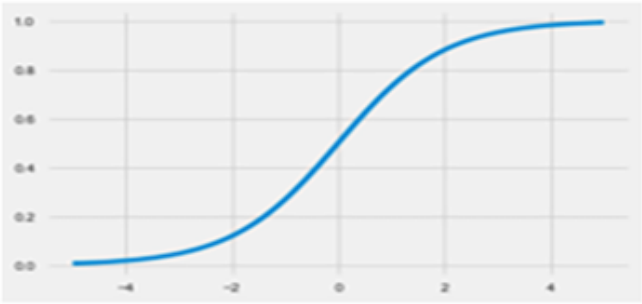
Les fonctions d'activation permettent aux modèles d'apprendre des relations plus complexes en capturant les non-linéarités dans les données.

- **Sigmoïde** : $\sigma(z) = \frac{1}{1+e^{-z}}$, plage de sortie (0, 1), utilisée pour la probabilité dans la classification binaire.
- **Tangente Hyperbolique (tanh)** : $\tanh(z)$, plage de sortie (-1, 1), version centrée et normalisée de la sigmoïde.
- **ReLU (Unité Linéaire Rectifiée)** : $f(z) = \max(0, z)$, non saturante, favorise la convergence rapide et atténue le problème de disparition des gradients pour les activations positives.
- **Leaky ReLU** : $f(z) = \max(\alpha z, z)$, variante de ReLU qui permet un petit gradient lorsque $z < 0$.
- **Softmax** : Utilisée pour la couche de sortie des problèmes de classification multi-classes.

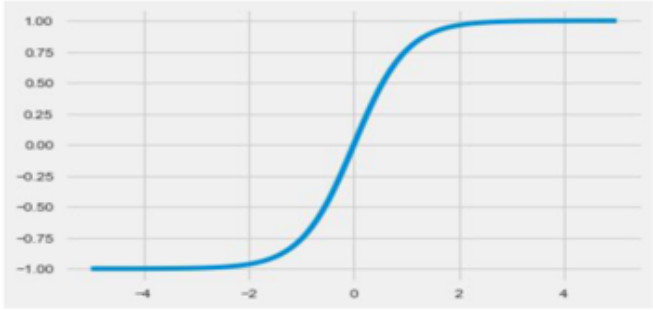
$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Illustration des fonctions d'activation

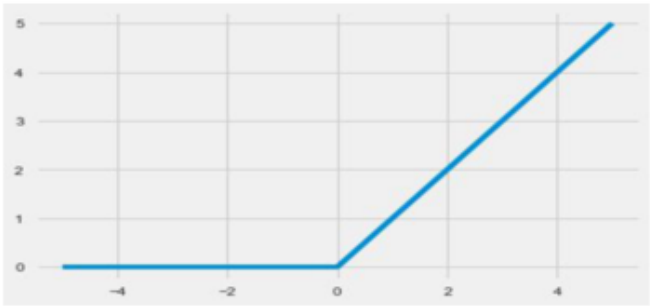
Sigmoïde



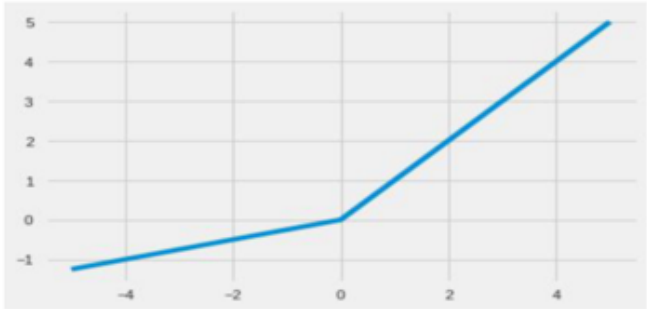
Tanh



ReLU



ReLU paramétrique



Entraînement d'un réseau de neurones artificiel

Principe d'Entraînement : L'entraînement d'un réseau de neurones consiste à ajuster ses poids pour minimiser une fonction de coût qui mesure l'erreur entre les prédictions et les vraies valeurs.

Descente de gradient stochastique (SGD) :

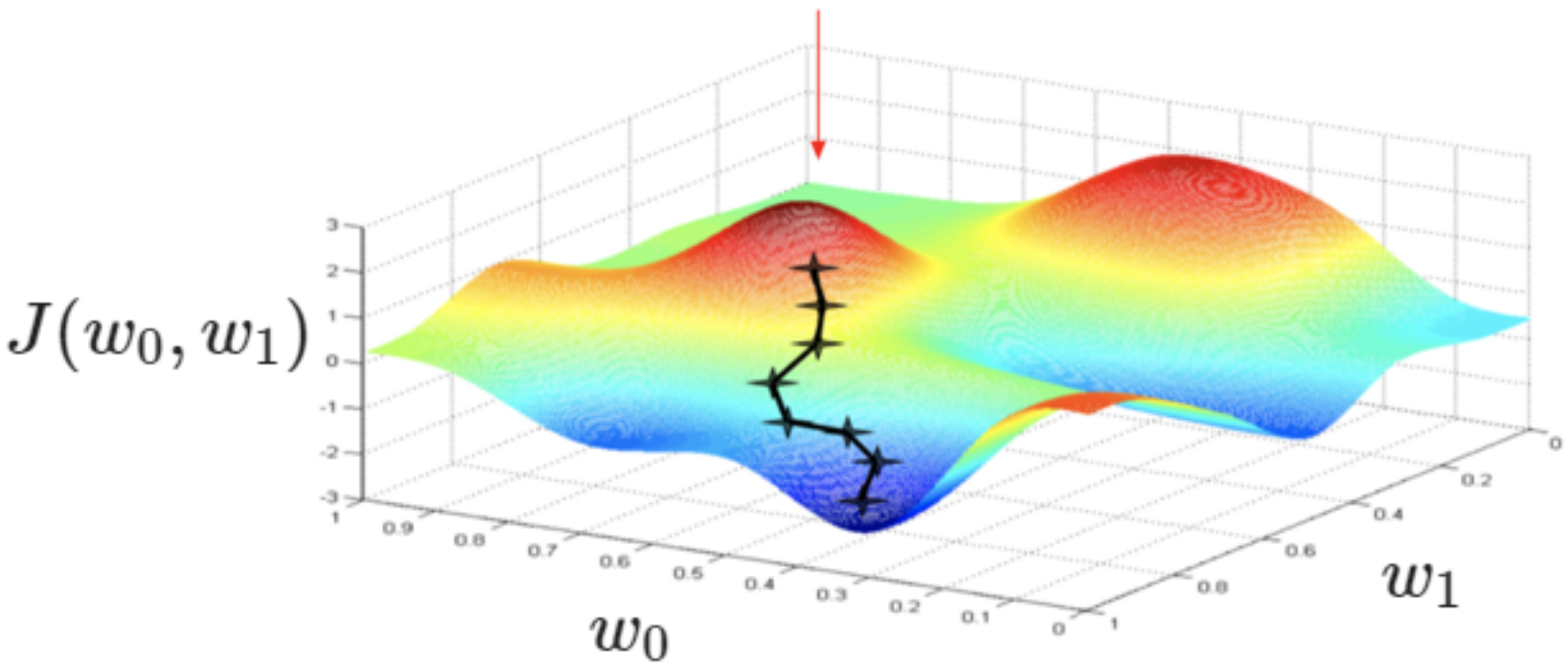
- Méthode d'optimisation utilisée pour mettre à jour les poids du réseau de manière itérative.
- À chaque itération, un sous-ensemble (batch) de données est utilisé pour calculer le gradient de la fonction de coût.
- Les poids sont mis à jour dans la direction opposée du gradient pour réduire l'erreur.

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \nabla_w J(w)$$

où :

- w_{old} et w_{new} sont les valeurs des poids avant et après la mise à jour,
- η est le taux d'apprentissage,
- $\nabla_w J(w)$ est le gradient de la fonction de coût par rapport aux poids.

Illustration graphique de la SGD

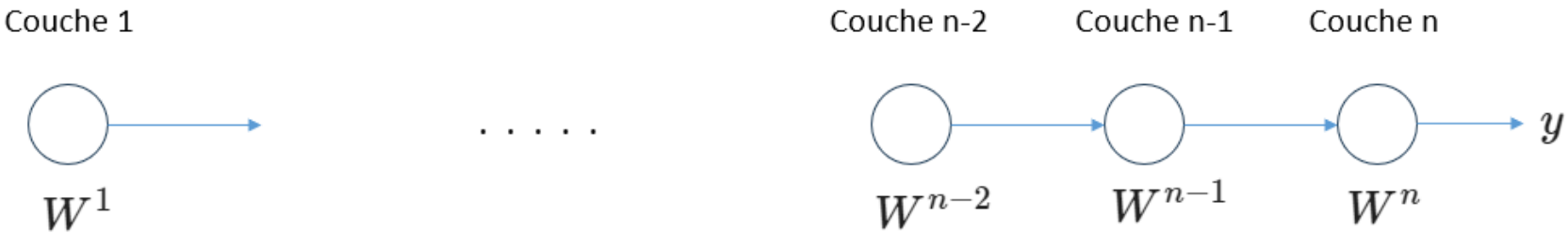


Entraînement des réseaux de neurones profonds

Dans les réseaux de neurones profonds, l'erreur calculée à la sortie du réseau dépend indirectement des poids des couches cachées. Ce lien indirect rend difficile de savoir comment ajuster ces poids pour réduire l'erreur.

Implications pour l'Entraînement :

- **Propagation de l'erreur** : Sans un mécanisme pour propager l'erreur de la sortie vers les couches antérieures, il est impossible de déterminer l'impact de chaque poids sur l'erreur finale.
- **Complexité de l'ajustement des Poids** : Chaque poids dans les couches cachées affecte l'erreur de sortie de manière complexe, nécessitant une méthode précise pour leur ajustement.



Rétropropagation du gradient

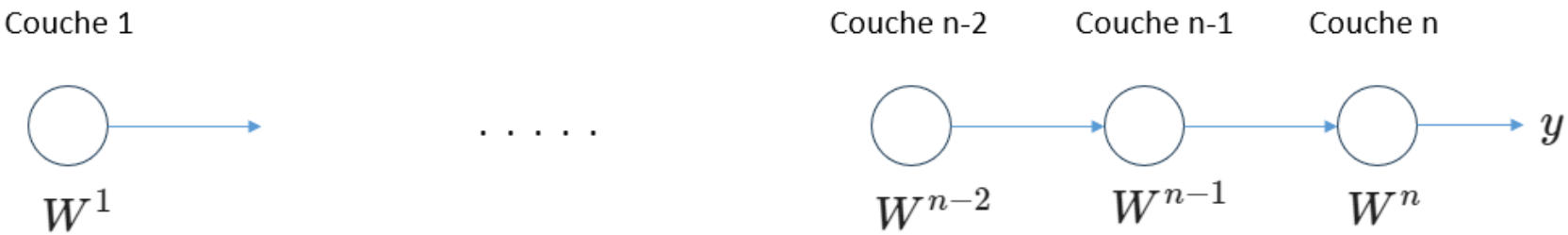
Il est facile de calculer les gradients correspondant à la dernière couche $\frac{\partial J(W^n)}{\partial W^n}$ car l'erreur J dépend immédiatement de W^n.

Examinons maintenant le cas de la couche $n-1$:

$$\frac{\partial J(W^n)}{\partial W^{n-1}} = \frac{\partial J(W^n)}{\partial O^n} \frac{\partial O^n}{\partial W^{n-1}}$$

Or

$$\frac{\partial O^n}{\partial W^{n-1}} = \frac{\partial O^n}{\partial O^{n-1}} \frac{\partial O^{n-1}}{\partial W^{n-1}}$$



Rétropropagation du gradient (backpropagation)

Principe de la Rétropropagation : La rétropropagation est un algorithme clé pour l'entraînement des réseaux de neurones profonds. Elle utilise la règle de la chaîne pour calculer les gradients de la fonction de coût par rapport à tous les poids du réseau.

Utilisation de la Règle de la Chaîne : La règle de la chaîne permet de décomposer le gradient de la fonction de coût par rapport à chaque poids en produits de dérivées partielles à travers les couches du réseau.

$$\frac{\partial J}{\partial w_{ij}} = \frac{\partial J}{\partial a_k} \cdot \frac{\partial a_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{ij}}$$

où a_k est la sortie activée du neurone, et z_k est son entrée pondérée.

Étapes de la Rétropropagation :

1. Propagation avant pour obtenir les activations à travers les couches.
2. Calcul de l'erreur à la couche de sortie.
3. Propagation de l'erreur en arrière en appliquant la règle de la chaîne pour obtenir les gradients.
4. Mise à jour des poids en utilisant les gradients calculés.

Batch normalization

La Batch normalization est une technique qui consiste à normaliser les entrées de chaque couche à travers un mini-batch. Elle est utilisée pour améliorer la vitesse, la performance et la stabilité des réseaux de neurones profonds.

- Normalise les activations de chaque couche pour avoir une moyenne proche de 0 et une variance proche de 1.
- Réduit le problème des gradients disparus en régulant la distribution des activations.
- Permet d'utiliser des taux d'apprentissage plus élevés et réduit la sensibilité aux initialisations des poids.

Équations de la batch normalization :

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mu_{\text{batch}}}{\sqrt{\sigma^2_{\text{batch}} + \epsilon}}$$

$$y^{(k)} = \gamma \hat{x}^{(k)} + \beta$$

où $x^{(k)}$ est l'activation d'entrée, μ_{batch} et σ^2_{batch} sont respectivement la moyenne et la variance du mini-batch, et γ et β sont des paramètres appris.

Initialisation des poids dans les réseaux de neurones

L'initialisation des poids dans un réseau de neurones a un impact significatif sur la vitesse et la qualité de la convergence lors de l'entraînement. Une mauvaise initialisation peut conduire à des problèmes tels que la disparition ou l'explosion des gradients.

- **Initialisation aléatoire** : Petites valeurs aléatoires pour briser la symétrie et permettre un apprentissage différencié.
- **Initialisation de Xavier/Glorot** : Conçue pour les fonctions d'activation sigmoïde/tanh, ajuste la variance en fonction du nombre de neurones entrants et sortants.
- **Initialisation de He** : Adaptée aux réseaux utilisant ReLU.

Formules d'Initialisation :

- Xavier/Glorot : $w \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}\right]$ ou $\mathcal{N}\left(0, \frac{2}{n_{\text{in}}+n_{\text{out}}}\right)$
- He : $w \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right]$ ou $\mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$

où n_{in} et n_{out} désignent les nombres d'entrées et de sorties de la couche ; $\mathcal{N}(0, \sigma^2)$ est paramétrée par sa variance.

Surapprentissage dans les réseaux de neurones

Le surapprentissage (overfitting) se produit lorsqu'un réseau de neurones apprend trop bien les détails et le bruit des données d'entraînement, au détriment de sa capacité à généraliser sur de nouvelles données.

Le surapprentissage dans les réseaux de neurones peut se produire pour différentes raisons :

- Complexité excessive du modèle
- Manque de diversité dans les données d'entraînement
- Entraînement prolongé

Stratégies pour atténuer le surapprentissage :

- **Régularisation** : Techniques de régularisation telles que L1/L2, Dropout, pour limiter la complexité du modèle.
- **Dropout** : "Eteindre" un ensemble de neurones choisis aléatoirement pendant l'entraînement.
- **Early Stopping** : Arrêt de l'entraînement lorsque la performance sur un ensemble de validation cesse de s'améliorer.
- **Augmentation de Données** : Augmenter la variété des données d'entraînement pour améliorer la robustesse du modèle.

Représentations distribuées

Introduction aux Embeddings

- **Qu'est-ce qu'un Embedding ?**
  - Un embedding est une représentation vectorielle d'un mot qui capture le contexte du mot dans un document, des relations sémantiques et syntaxiques avec d'autres mots.
  - Ils transforment des mots en vecteurs de nombres pour que les algorithmes de machine learning puissent les traiter efficacement.
- **Pourquoi sont-ils importants ?**
  - Les embeddings permettent de capturer non seulement l'identité d'un mot mais aussi ses aspects sémantiques et contextuels.
  - Ils facilitent des tâches telles que la classification de texte, la traduction automatique, et la détection des sentiments.
- **Évolution des embeddings**
  - Historiquement, les mots étaient représentés comme des indices ou des vecteurs one-hot, où chaque mot est indépendant des autres.
  - Les embeddings modernes, tels que Word2Vec et GloVe, représentent les mots dans des espaces vectoriels continus où les mots de sens similaires sont proches les uns des autres.

Différence entre One-hot Encoding et Embeddings

- **One-hot Encoding**
  - Chaque mot est représenté par un vecteur avec un '1' dans la position qui lui est propre et des '0' partout ailleurs. Ce type de représentation est simple mais très inefficace en termes d'espace et ne capture pas les relations entre les mots.
  - Exemple : pour un vocabulaire de dimension 100 000 :

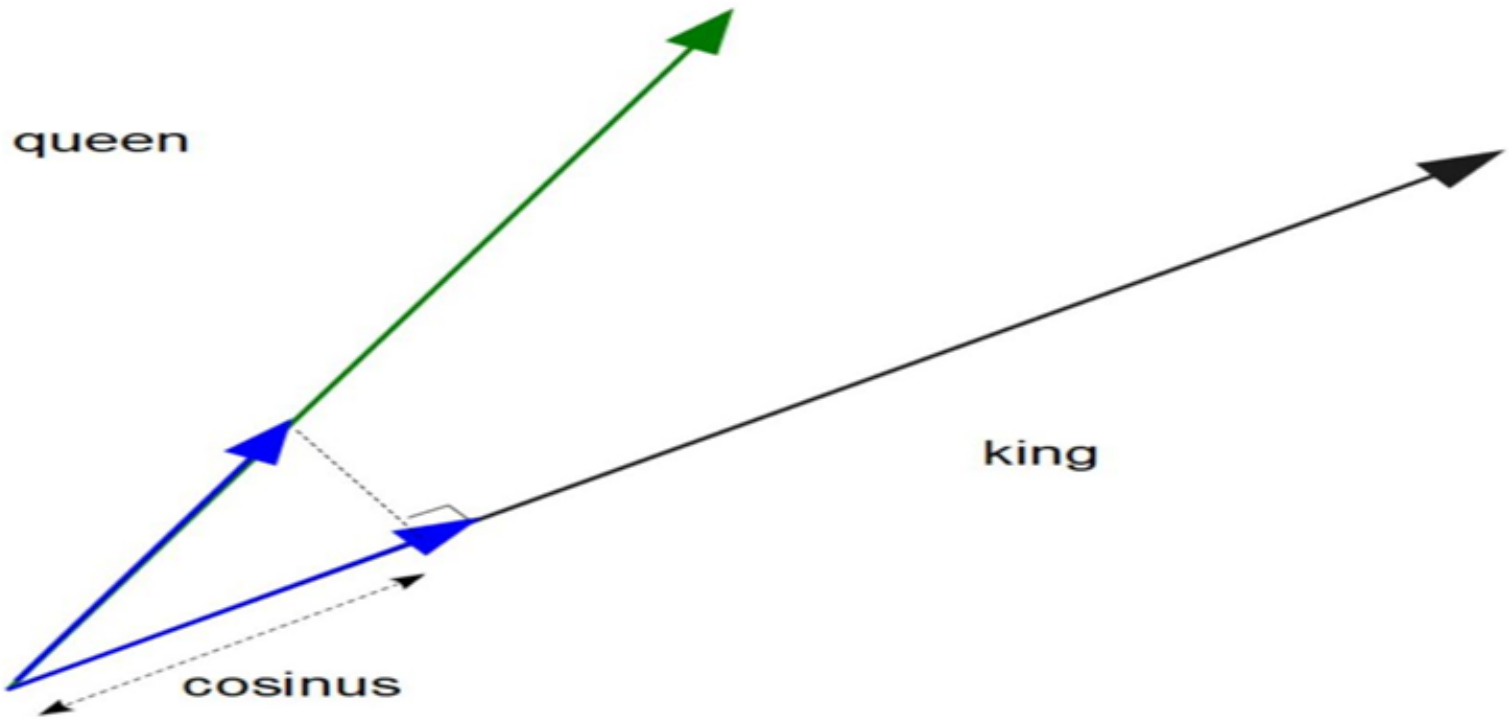
    Véhicule = [0, 0, 1, 0, 0, 0, 0, 0, 0,. . . ,0 ,0]
    Voiture = [0, 0, 0, 0, 0, 1, 0, 0, 0,. . . ,0 ,0]

    Cette representation ne permet pas de prendre en compte la dimension sémantique

- **Embeddings**
  - Les embeddings représentent les mots comme des vecteurs denses de nombres flottants (généralement entre 50 et 300 dimensions).
  - Cette représentation est beaucoup plus riche et peut capturer des relations complexes entre les mots, comme la similarité sémantique.

Similarité sémantique

Nous sommes intéressés par des représentations de mots qui capturent la distance sémantique, sous forme de produit scalaire par exemple (ou distance cosine).



Représentations distribuées : Principes

"You shall know a word by the company it keeps"

— J.R. Firth (1957)

Les mots sont similaires s'ils apparaissent fréquemment dans le même contexte.

- Il conduit son *véhicule* pour rentrer à la maison.
- Il conduit sa *voiture* pour rentrer chez lui.

Construction d'une représentation distribuée

Considérons le corpus suivant à titre d'exemple : **cnn in crop analysis**

cnn and svm are widely used.
linear_regression performed along with svm
linear_regression for crop and farm
svm being used for farm monitoring
Do cnn, svm and linear_regression appear in the same context?

Le vocabulaire est alors :

```
[cnn, in, crop, analysis, and, svm, are, widely, used,
linear_regression, performed, along, with, for, farm, being,
monitoring, do, appear, the, same, context]
```

$$\dim(\text{vocabulaire}) = 22$$

Similarité sémantique

La matrice de co-occurrence représente la fréquence à laquelle les mots apparaissent ensemble deux à deux.

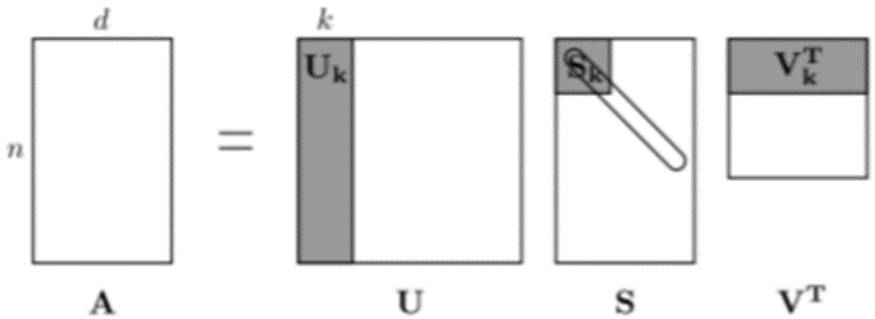
	cnn	in	crop	...
cnn	0	2	1	1	1	2	1	1	1	1	0	0	0	0	0	0	0	1	1	1	1	1
in	2	0	1	1	0	1	0	0	0	1	0	0	0	0	0	0	0	1	1	1	1	1
crop	1	1	0	1	1	0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	0
...	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	1	0	1	0	0	1	1	1	1	1	0	0	0	1	1	0	0	0	0	0	0	0
	2	1	0	0	1	0	1	1	2	2	1	1	1	1	1	1	1	1	1	1	1	1
	1	0	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
	1	0	0	0	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
	1	0	0	0	1	2	1	1	0	0	0	0	0	1	1	1	1	0	0	0	0	0
	1	1	1	0	1	2	0	0	0	0	1	1	1	1	1	0	0	1	1	1	1	1
	0	0	0	0	0	1	0	0	0	1	0	1	1	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	1	0	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	1	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0
	0	0	1	0	1	1	0	0	1	1	0	0	0	0	2	1	1	0	0	0	0	0
	0	0	1	0	1	1	0	0	1	1	0	0	0	2	0	1	1	0	0	0	0	0
	0	0	0	0	0	1	0	0	1	0	0	0	0	1	1	0	1	0	0	0	0	0
	0	0	0	0	0	1	0	0	1	0	0	0	0	1	1	1	0	0	0	0	0	0
	1	1	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	1	1	1	1
	1	1	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	1	0	1	1	1
	1	1	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	1	1	0	1	1
	1	1	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	1	1	1	0	1
	1	1	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	1	1	1	1	0

Réduction de la dimension

La décomposition en valeurs singulières est une technique permettant de réduire la dimension des données (similaire à l'ACP).

Étant donné une matrice $A \in \mathbb{R}^{n \times d}$

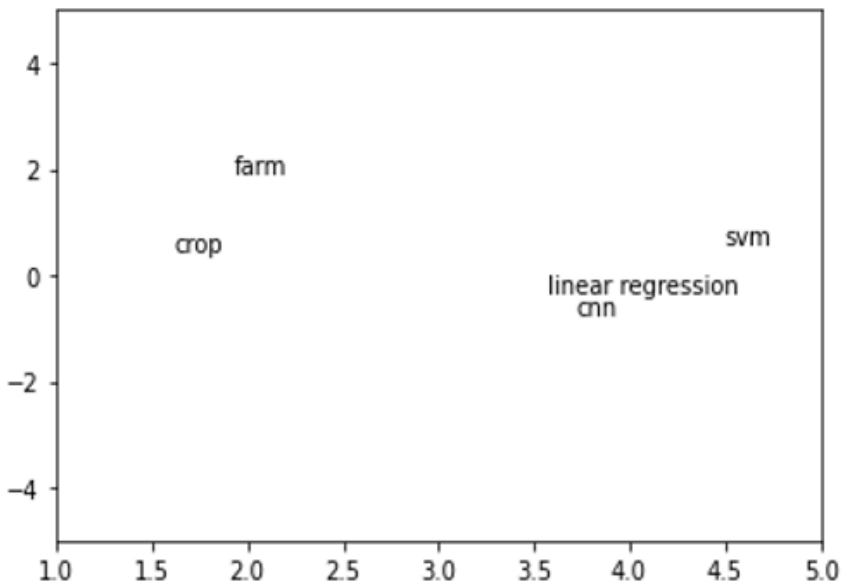
$$A = UDV^T \quad \text{où} \quad U \in \mathbb{R}^{n \times r}, D \in \mathbb{R}^{r \times r}, V \in \mathbb{R}^{d \times r}.$$



U est la matrice contenant les représentations (vecteurs de mots)

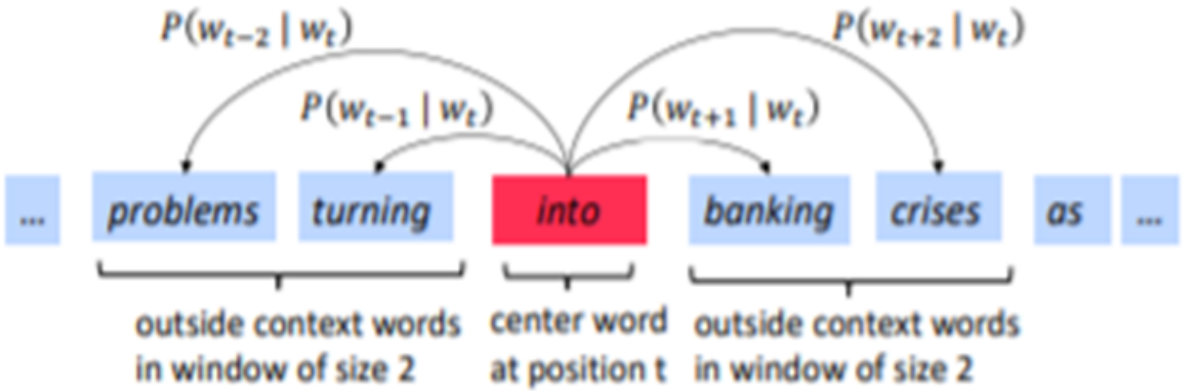
Vsiualisation des représentations distribuées

```
cnn [[ 3.72113518, -0.73233585],
In   [ 2.7940387 , -1.40864784],
crop [ 1.61178082,  0.44786021],
...  [ 0.77784994, -0.31230389],
     [ 2.12245048,  1.29571556],
     [ 4.49293777,  0.58090417],
     [ 1.33312748,  0.66758743],
     [ 1.33312748,  0.66758743],
     [ 2.25882809,  1.80738544],
     [ 3.56593605, -0.31006976],
     [ 0.95394179,  0.079159  ],
     [ 0.95394179,  0.079159  ],
     [ 0.95394179,  0.079159  ],
     [ 1.92921553,  1.94783497],
     [ 1.92921553,  1.94783497],
     [ 1.12301312,  1.42126096],
     [ 1.12301312,  1.42126096],
     [ 2.26025282, -1.31571175],
     [ 2.26025282, -1.31571175],
     [ 2.26025282, -1.31571175],
     [ 2.26025282, -1.31571175],
     [ 2.26025282, -1.31571175]]
```

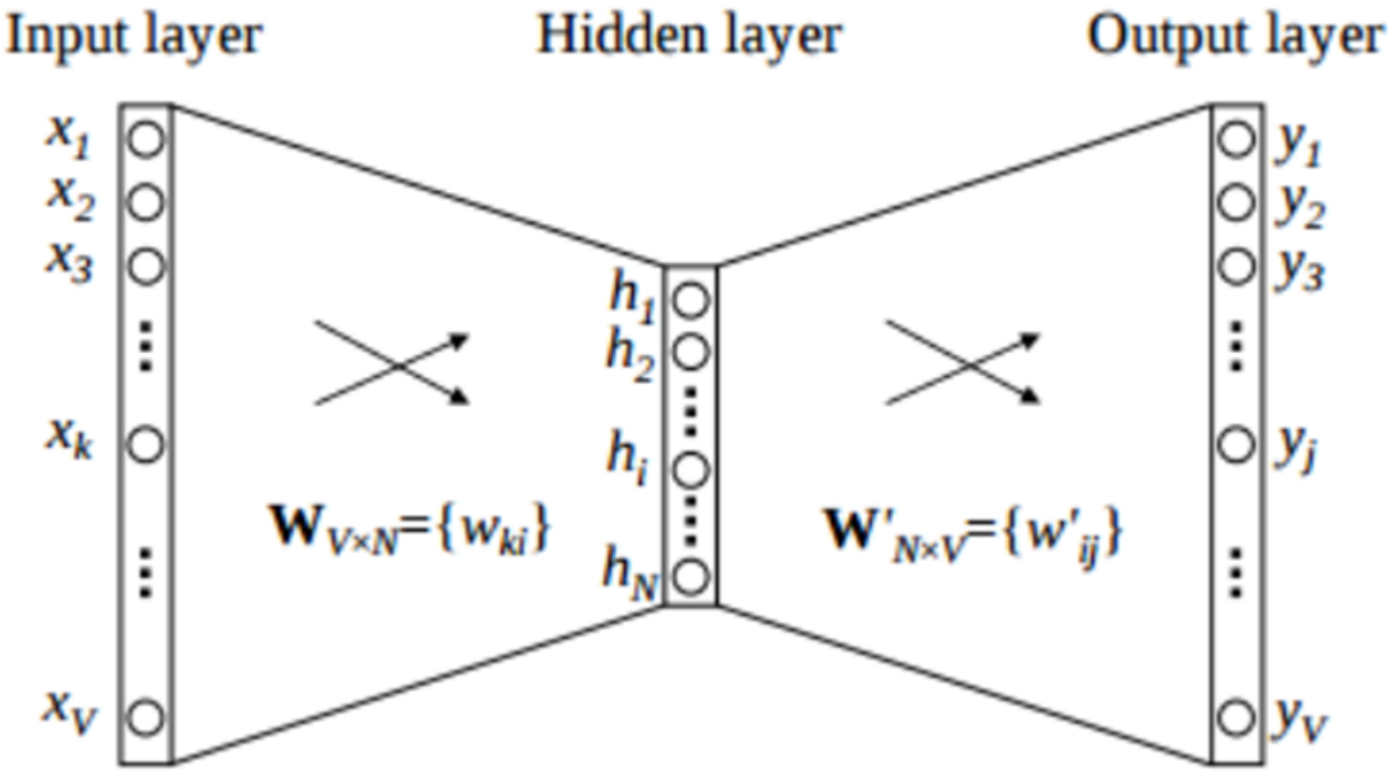


Word2Vec : Principes

- **Principes de base :**
  - Word2Vec est basé sur l'hypothèse distributionnelle : les mots qui apparaissent dans des contextes similaires ont des significations similaires.
  - Utilise un réseau de neurones peu profond pour apprendre des embeddings de mots à partir de grands corpus.
- **Deux architectures principales :**
  1. *Continuous Bag of Words (CBOW)* : Prédit le mot cible à partir du contexte.
  2. *Skip-Gram* : Prédit le contexte à partir du mot cible.



Architecture du modèle CBOW



Modèle CBOW : Formulation mathématique

- **Objectif :** Prédire le mot cible w_t en fonction du contexte C.
- **Fonction de prédiction :**

$$P(w_t|C) = \frac{e^{\mathbf{v}_{w_t}^T \cdot \mathbf{h}}}{\sum_{w \in W} e^{\mathbf{v}_w^T \cdot \mathbf{h}}} \tag{1}$$

- **Où :**
  - $\mathbf{v}_w$ est le vecteur de l'embedding du mot w.
  - $\mathbf{h}$ est le vecteur du contexte, la moyenne des embeddings des mots du contexte.
  - W est l'ensemble de tous les mots du vocabulaire.
- **Optimisation :** Minimisation de la fonction de coût, souvent une forme de cross-entropy ou log-likelihood négatif.

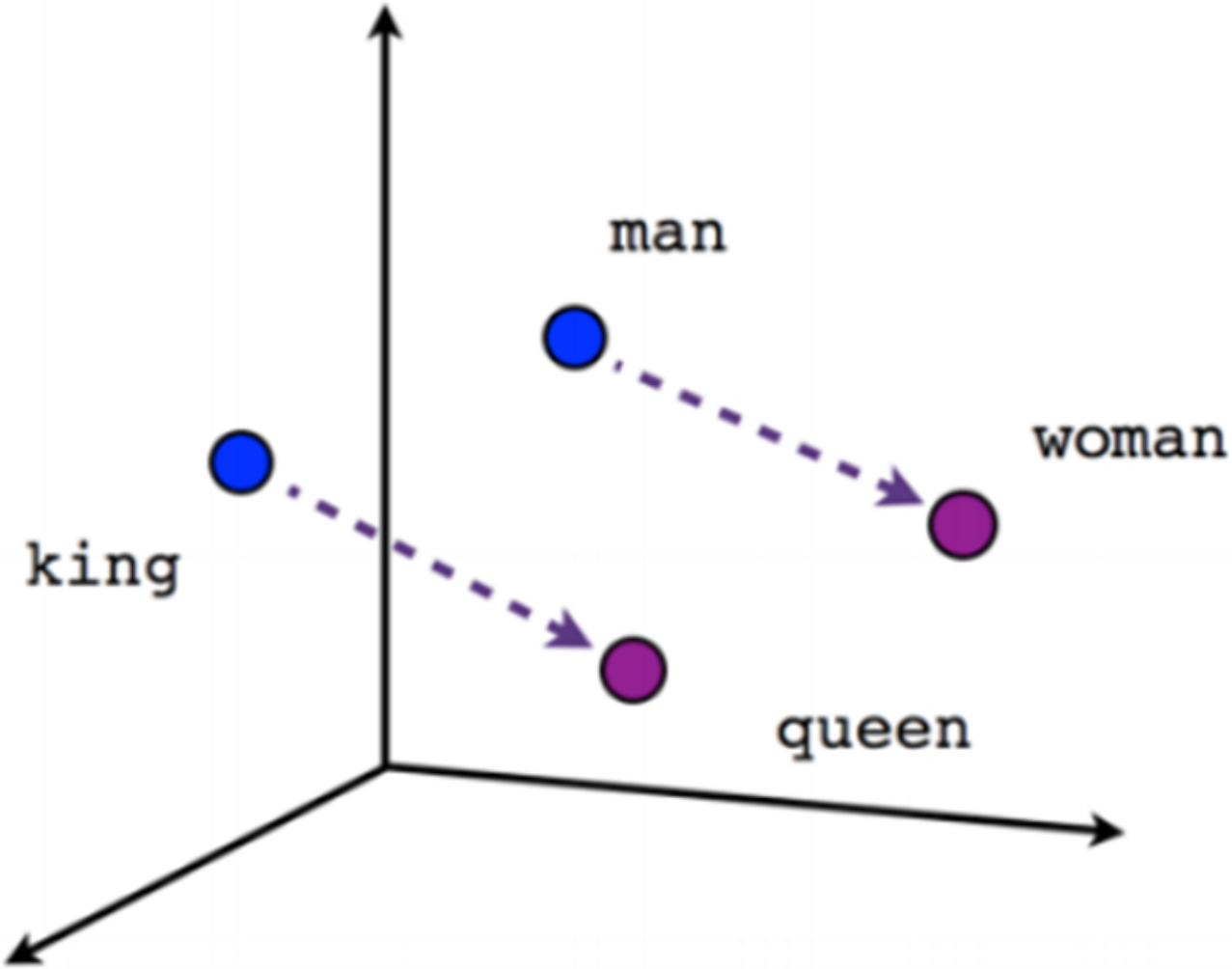
Modèle Skip-Gram : Formulation mathématiques

- **Objectif :** Prédire les mots de contexte à partir du mot cible w_t.
- **Fonction de prédiction :**

$$P(C|w_t) = \prod_{w_c \in C} \frac{e^{\mathbf{v}_{w_c}^T \cdot \mathbf{h}}}{\sum_{w \in W} e^{\mathbf{v}_w^T \cdot \mathbf{h}}} \tag{2}$$

- **Où :**
  - $\mathbf{v}_{w_c}$ est le vecteur de l'embedding du mot de contexte w_c.
  - $\mathbf{h}$ est le vecteur de l'embedding du mot cible w_t.
- **Optimisation :** Minimisation de la fonction de coût, souvent une forme de cross-entropy ou log-likelihood négatif.

Représentations Word2Vec



Applications pratiques de Word2Vec

- **Analogie de mots :**
  - Word2Vec est célèbre pour capturer des relations complexes, comme "homme est à femme ce que roi est à reine".
  - Permet de résoudre des analogies en utilisant des opérations arithmétiques simples sur les vecteurs de mots.
- **Clustering sémantique :**
  - Les embeddings peuvent être utilisés pour regrouper des mots sémantiquement similaires, facilitant l'analyse de textes volumineux.
- **Amélioration des systèmes de recommandation :**
  - Les vecteurs de mots de Word2Vec peuvent être utilisés pour améliorer la précision des recommandations en comprenant mieux les préférences des utilisateurs.

Autres représentations distribuées

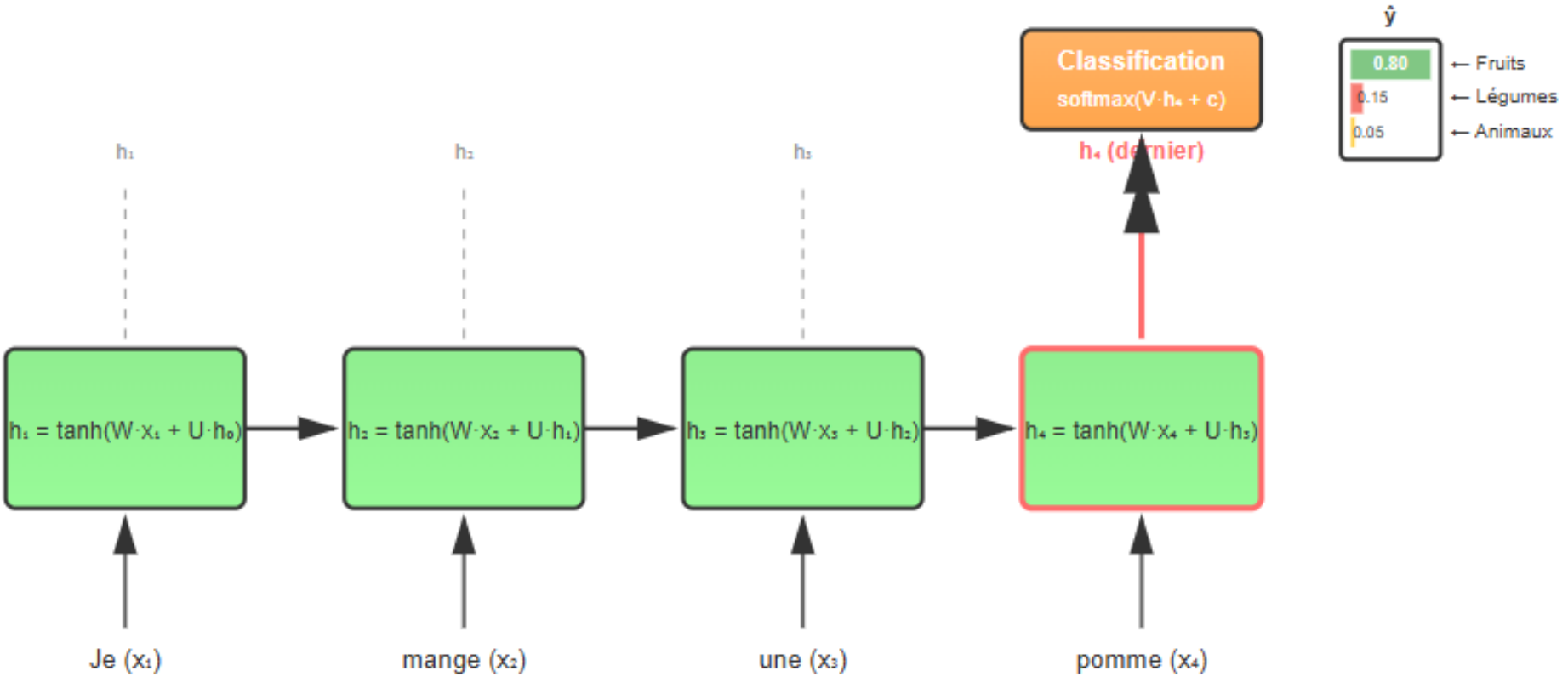
Il existe d'autres répsentations distribuées :

- Glove (2014) : proposé par une équipe de Stanford, il combine à la fois les techniques modernes de deep learning et les techniques statistiques (co-occurrences entre les mots). Il permet d'avoir des représentations des mots plus globales que Word2Vec.
- Fasttext (à partir de 2016) : la librairie a été créée par Facebook. Le modèle est entraîné sur des subwords (n-grams de caractères). Il est ainsi plus efficace pour traiter les mots inconnus (out of vocabulary).
- Représentations contextuelles (Transformers), etc.

Large Language Models (LLMs)

Anciens modèles pour le traitement du langage : RNN

- **Mémoire à court terme** : Les reaseaux de neurones récurrents (RNN) ont la capacité de se "souvenir" d'informations passées grâce à leurs connexions récurrentes.
- **Traitement de séquences** : Ils sont particulièrement adaptés pour des tâches où les données sont séquentielles et où le contexte est important.



Transformers

arXiv:1706.03762v7 [cs.CL] 2 Aug 2023

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

†Work performed while at Google Brain.

‡Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

Transformer originel

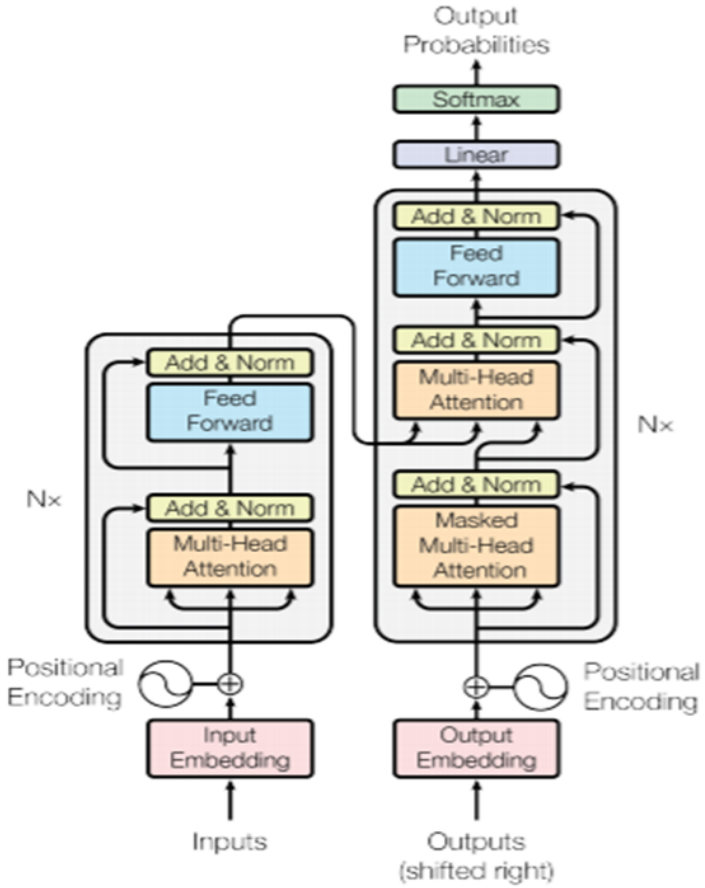
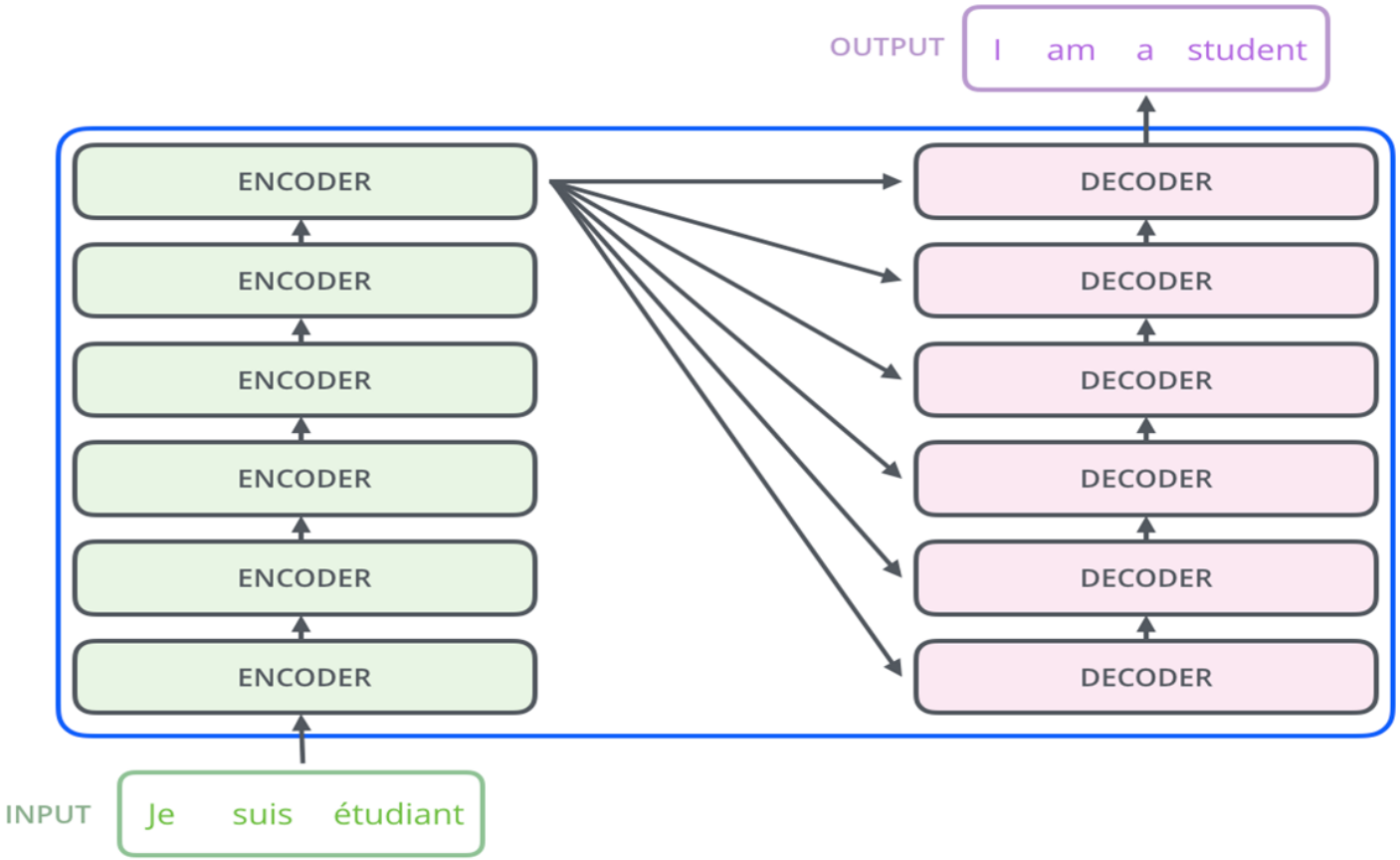
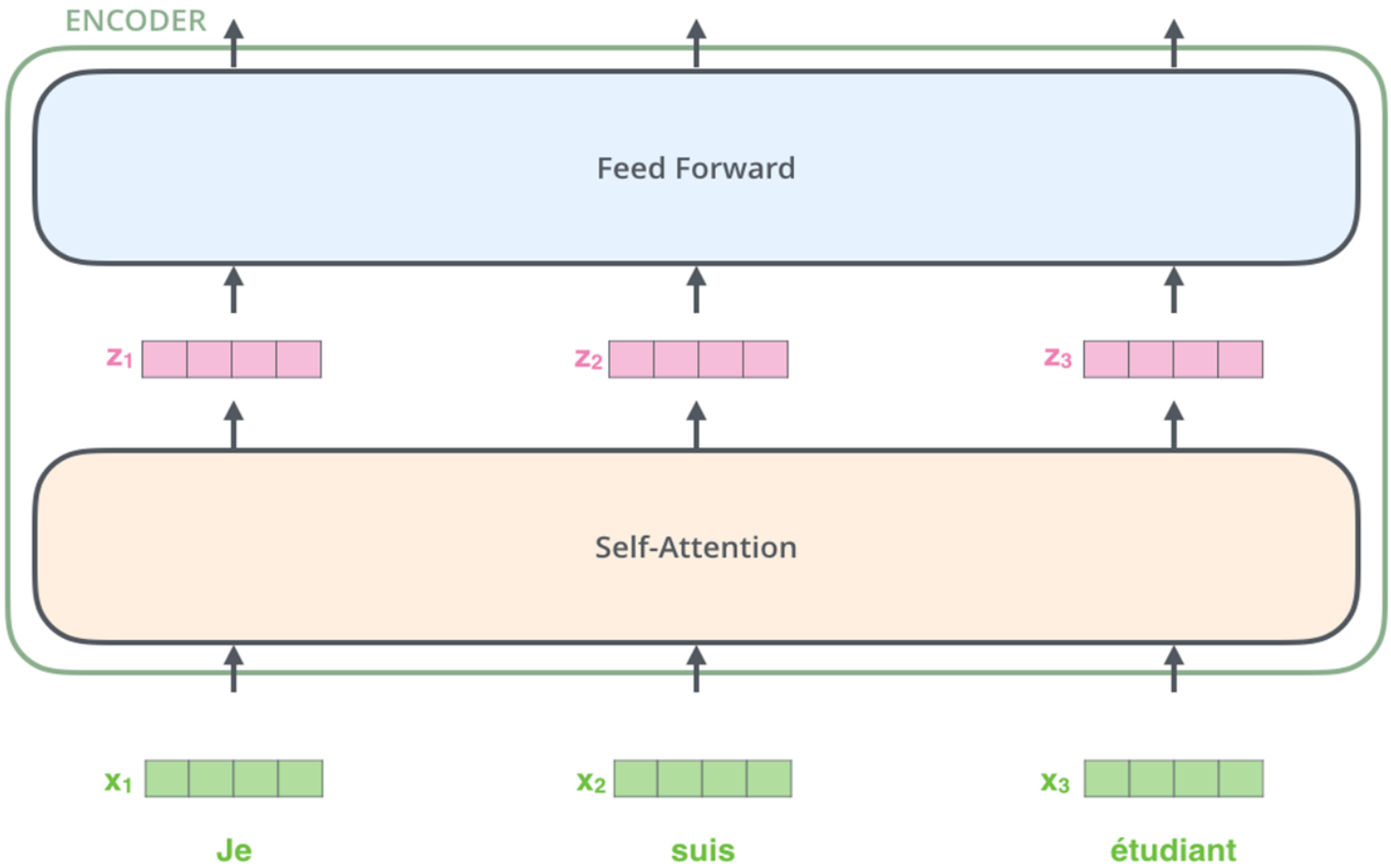


Figure 1: The Transformer - model architecture.

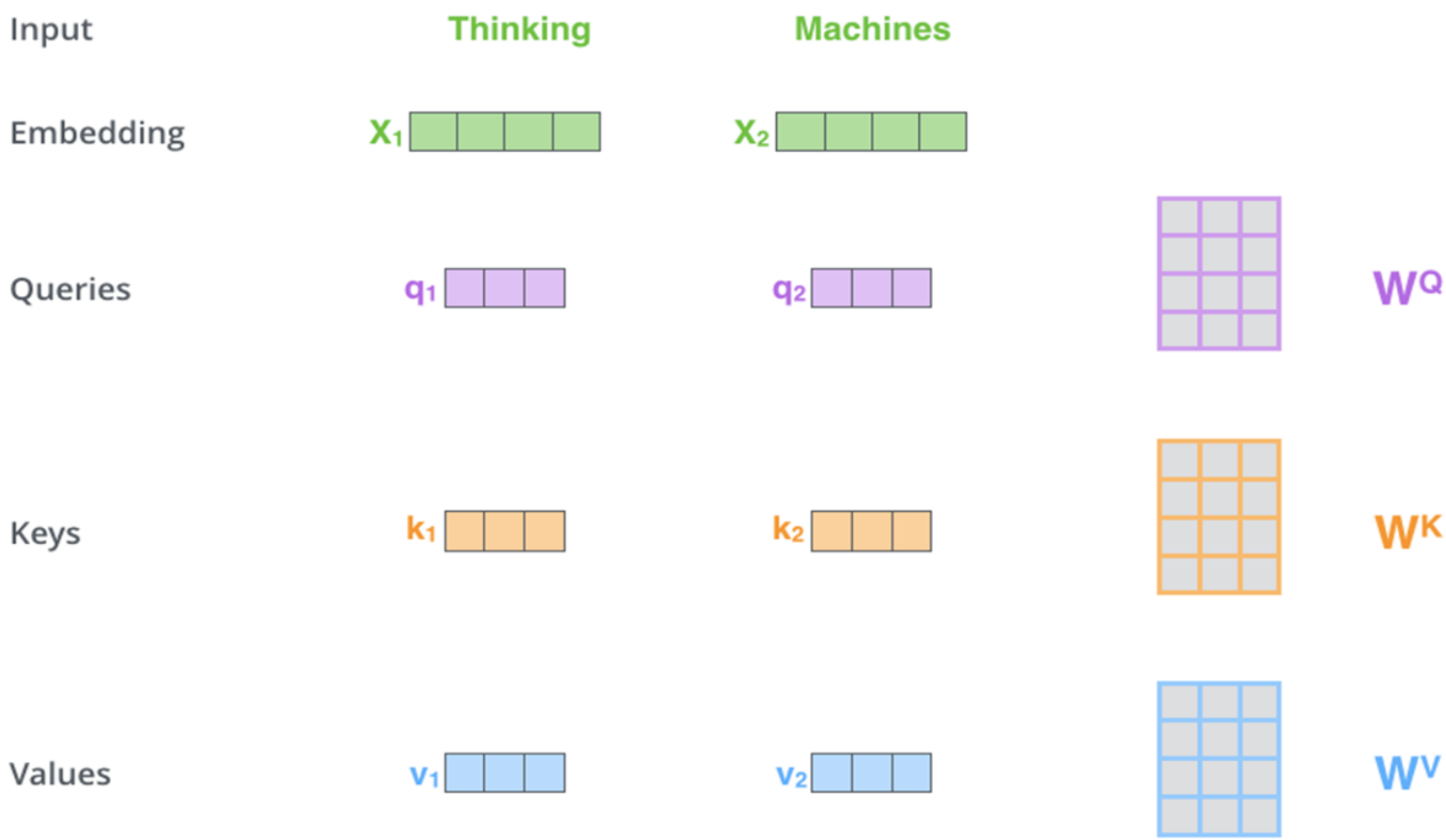
Mécanisme de cross-attention



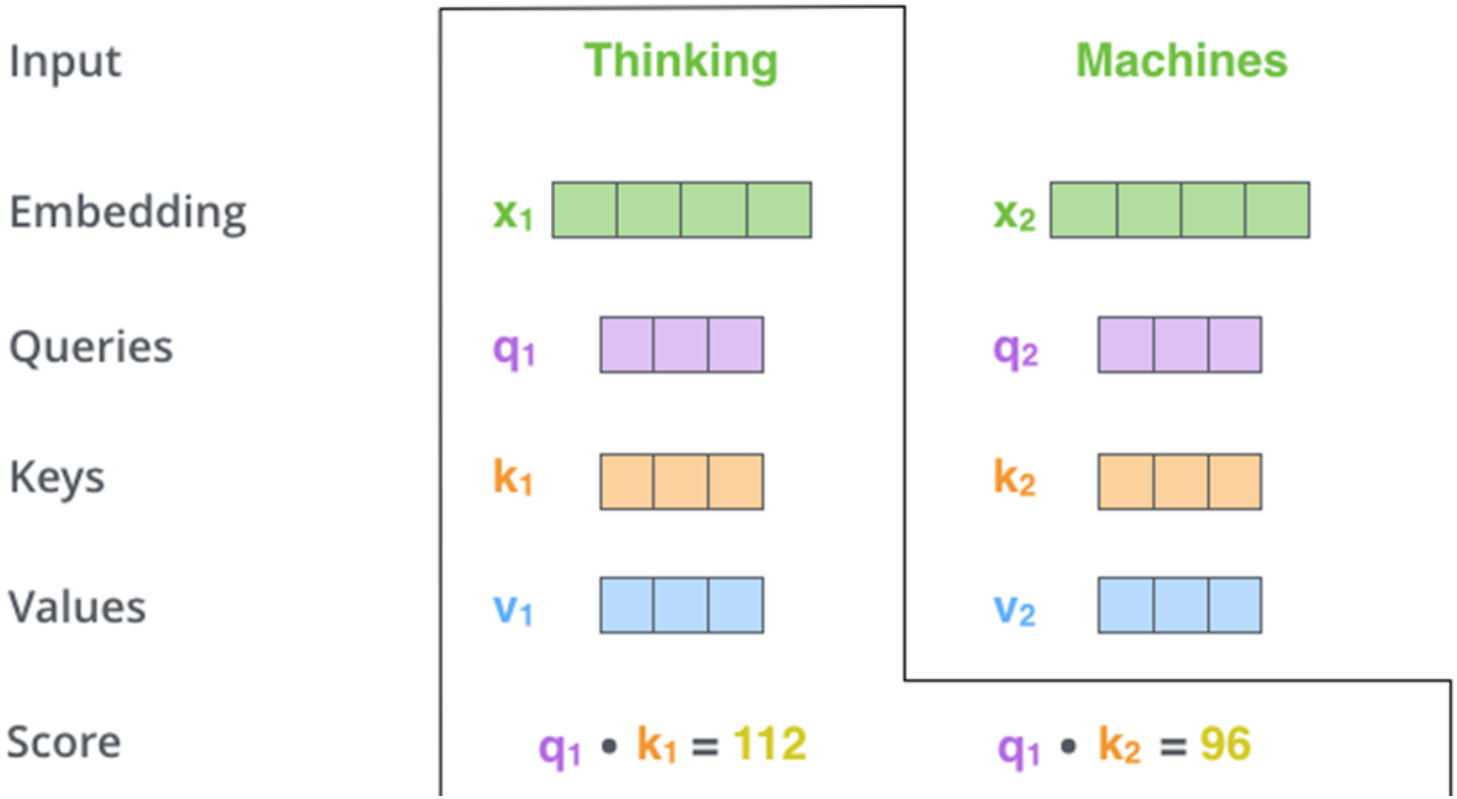
Mécanisme de self-attention



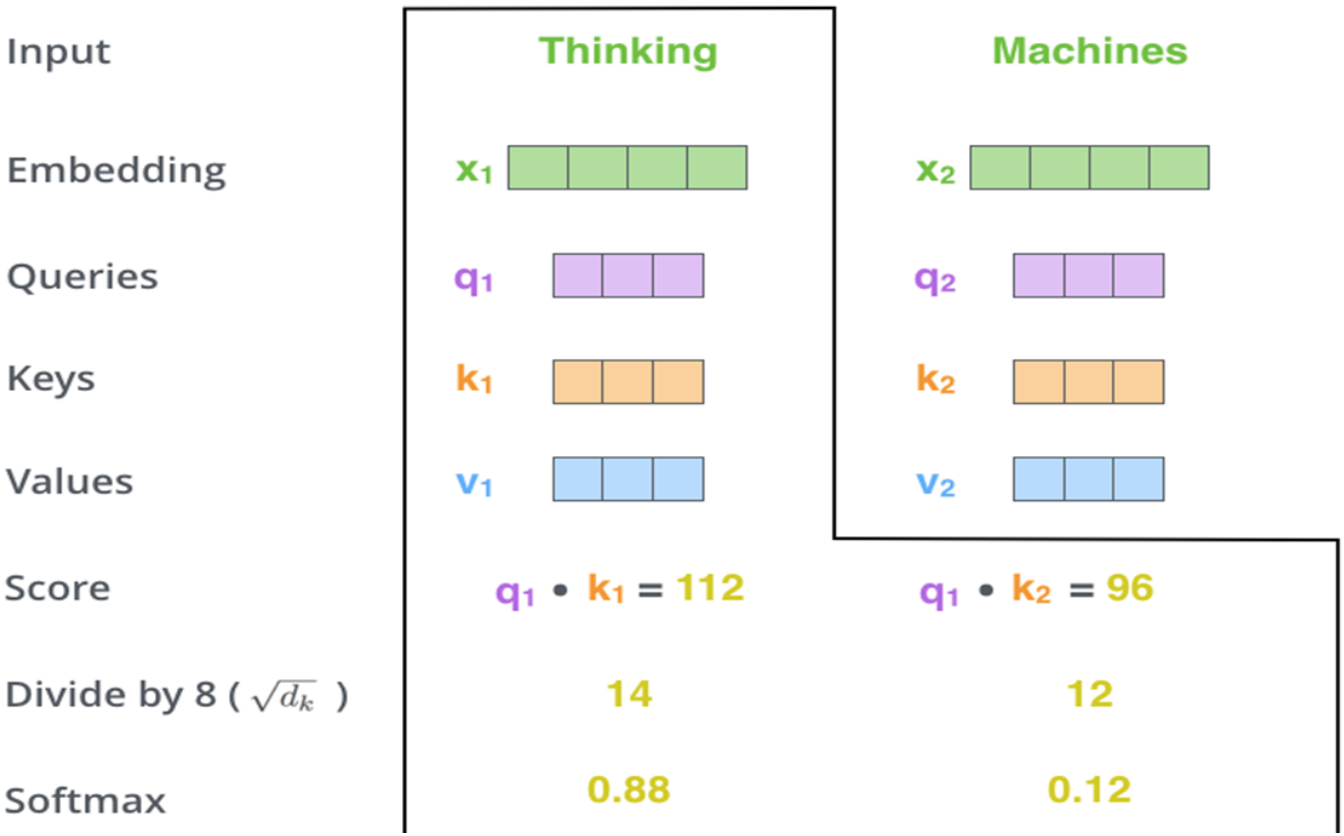
Fonctionnement du mécanisme de self-attention



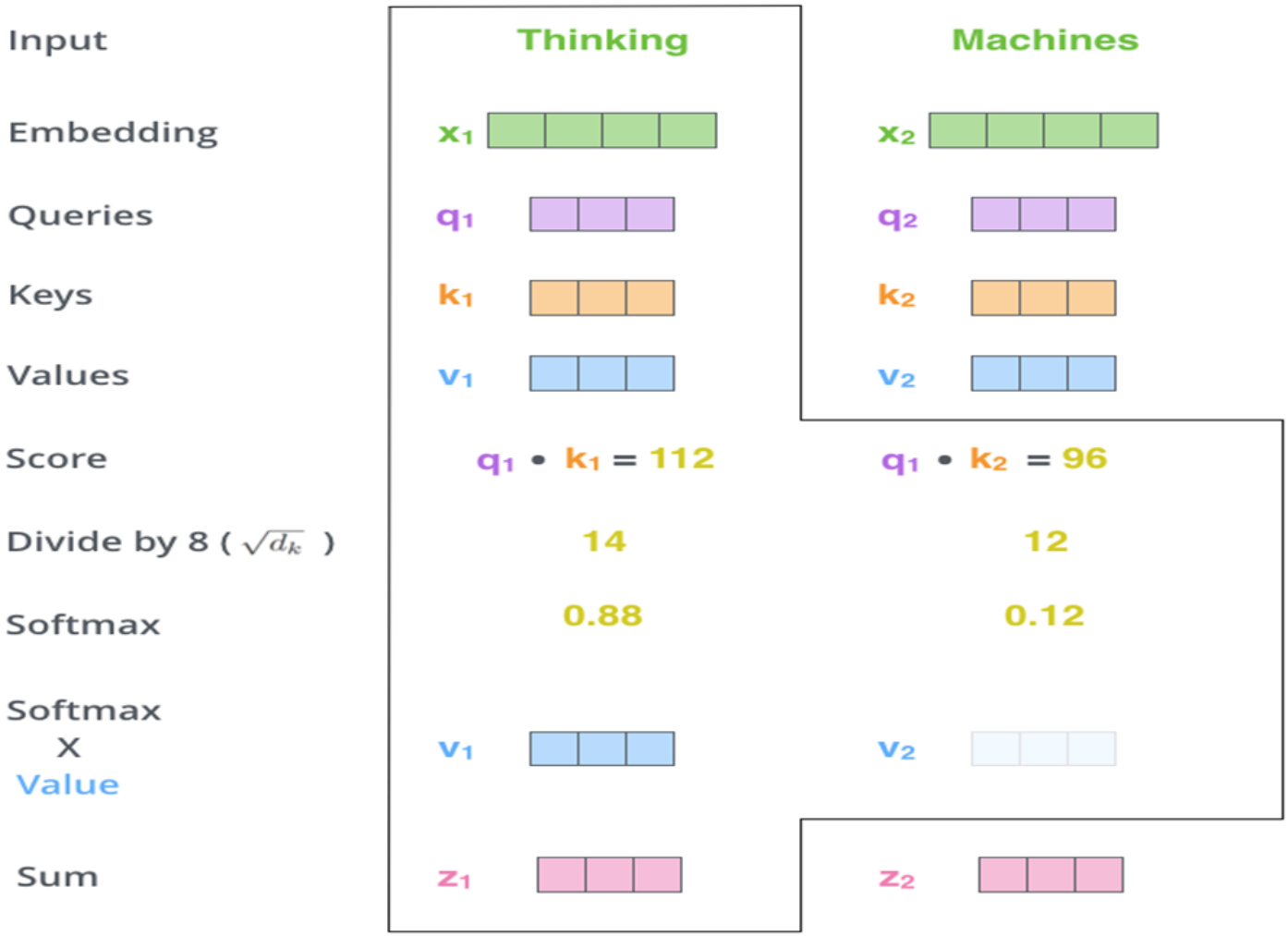
Calcul des scores de self-attention



Calcul des scores de self-attention



Calcul de la nouvelle représentation



Performances du tranformer originel

Entraîné sur WMT 2014 English- German dataset, comprenant près de 4.5 millions de phrases sentence, le WMT 2014 English-French dataset, comprenant près de 36 millions de phrases.

- 8 NVIDIA P10 GPUs
- **Base** : 12 heures
- **Large** : 3.5 jours

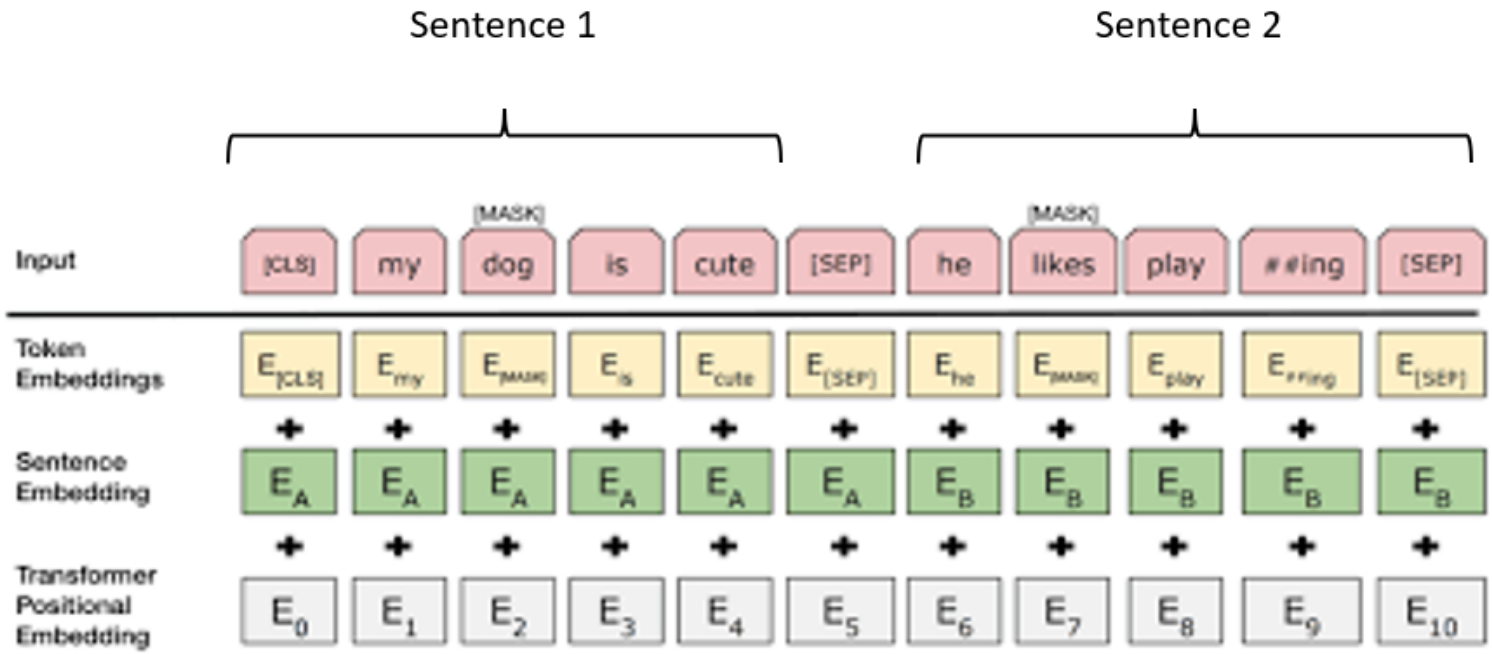
Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

BERT

BERT (Bidirectional Encoder Representations from Transformers) représente une avancée majeure dans le NLP, introduisant une approche novatrice pour la modélisation de langage. Il utilise la bidirectionnalité pour comprendre le contexte des mots, permettant une compréhension plus fine du langage.

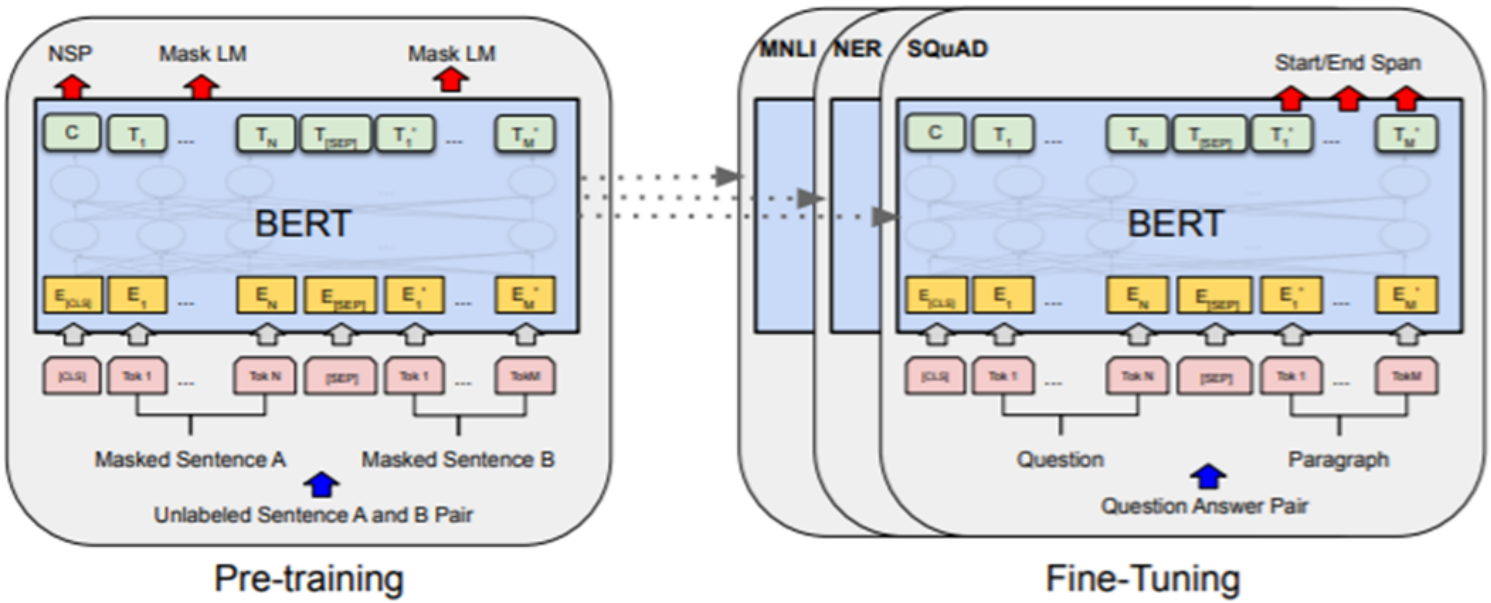
Pré-entraînement



Fine-tuning de BERT pour des tâches spécifiques

Après pré-entraînement, BERT est affiné pour des tâches spécifiques:

- Ajout d'une couche de sortie spécifique à la tâche (classification, NER, QA).
- Fine-tuning de tous les paramètres du modèle pré-entraîné sur le corpus de la tâche.



Performances de BERT

BERT a été pré-entrainé sur le BookCorpus (800 millions de mots) et English Wikipedia (2,500 millions de mots).

Deux modèles :

- **BERT Base** : 12 couches avec 110 millions de paramètres.
- **BERT Large** : 24 couches avec 340 millions de paramètres.

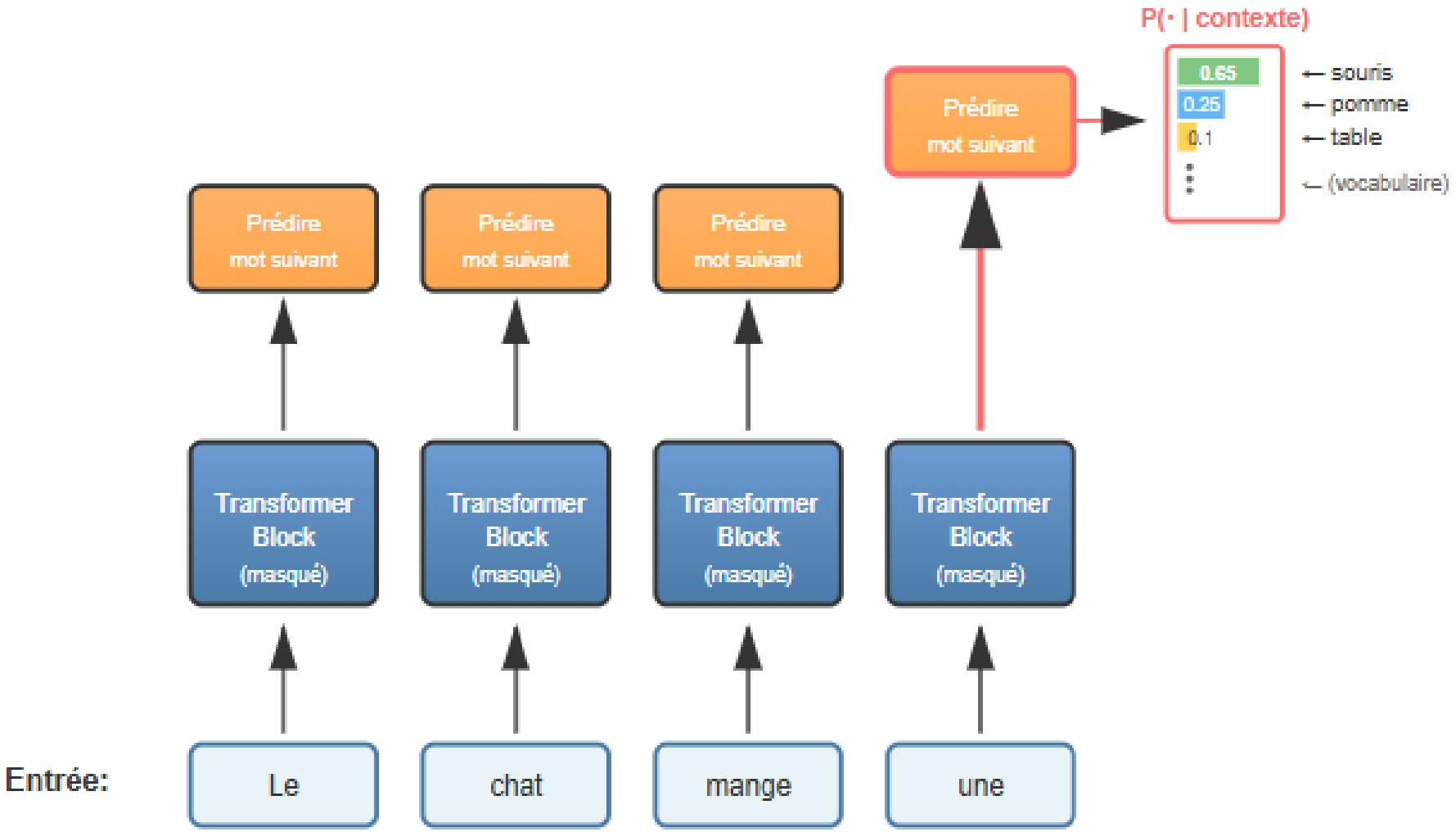
System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

Modèles génératifs (GPT)

« Ce que je ne peux pas créer,
je ne le comprends pas. »

– Richard Feynman

Fonctionnement des modèles génératifs



Le paramètre de température

Définition La température (T) contrôle le degré d'aléa dans les réponses générées. Elle agit sur la distribution de probabilité des mots avant l'échantillonnage.

Formule mathématique :

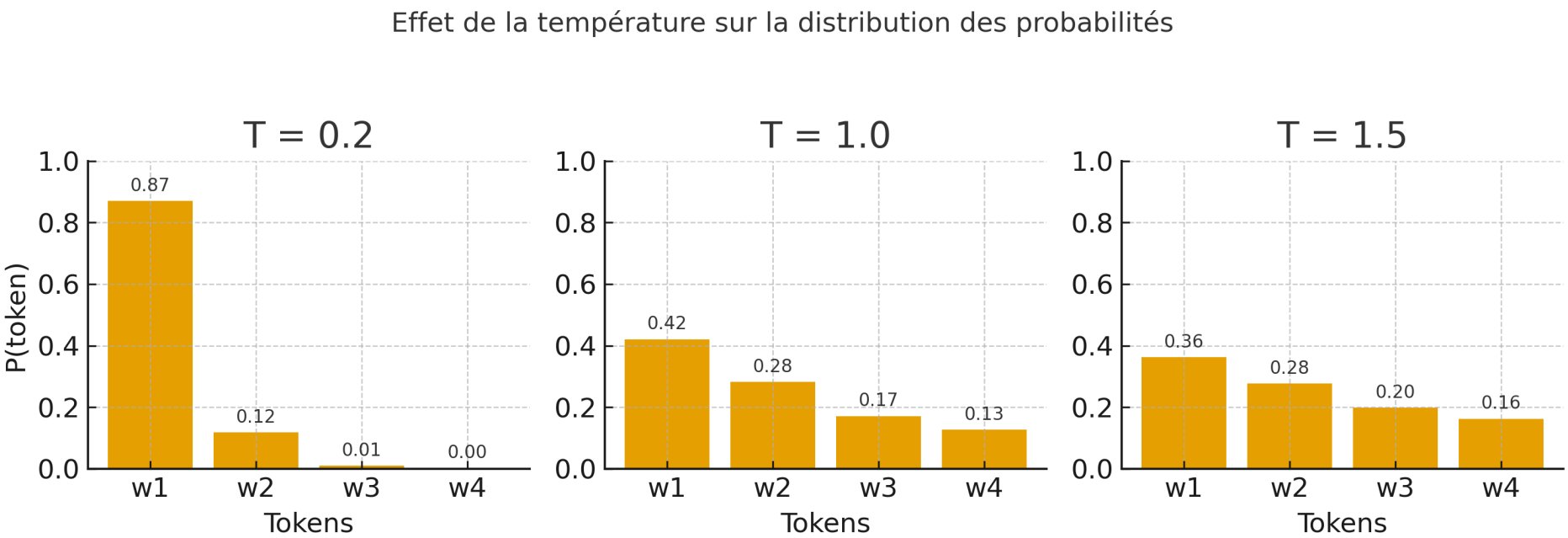
$$P(x_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

- $T < 1 \rightarrow$ distribution plus concentrée, réponses plus prévisibles
- $T > 1 \rightarrow$ distribution plus plate, réponses plus variées et créatives

Autrement dit :

- Basse température $\rightarrow$ précision et cohérence.
- Haute température $\rightarrow$ imagination et diversité.

Illustration graphique de l'effet de la température



La tokénisation

Problème de départ : Les modèles de langage ne peuvent pas traiter directement du texte brut : les mots, les accents, la ponctuation et les variations sont trop nombreux (plusieurs millions de combinaisons possibles).

La tokénisation : Elle consiste à découper le texte en **unités réutilisables et limitées en nombre**, appelées *tokens*. Chaque token est ensuite représenté par un identifiant numérique que le modèle peut apprendre à prédire.

Intérêt de la tokénisation :

- Réduire la complexité du vocabulaire : on passe de millions de mots à quelques dizaines de milliers de tokens.
- Gérer les mots inconnus ou rares en les décomposant en sous-unités connues.
- Offrir une représentation numérique uniforme du langage, compatible avec les réseaux de neurones.

La composition des données d'entraînement des modèles

Principe général Les modèles de langage sont entraînés sur d'immenses corpus de texte — souvent plusieurs milliers de milliards de tokens — issus de sources très diverses pour couvrir la langue, les faits et les styles.

Sources typiques de données :

- **Web public** : pages web, articles, forums, Wikipédia (ex. Common Crawl).
- **Livres et documents** : œuvres littéraires, articles scientifiques, rapports techniques.
- **Données de code** : dépôts GitHub, notebooks, documentation technique.
- **Conversations et dialogues** : jeux de données de chat, discussions de forums, données annotées par des humains.
- **Données synthétiques** : textes générés par d'autres modèles pour enrichir certaines thématiques.

Équilibre recherché :

- Diversité linguistique et thématique (langues, registres, domaines).
- Qualité et cohérence (filtrage du spam, des doublons, des contenus toxiques).
- Représentation équilibrée entre langage courant, technique et conversationnel.

GPT-3

- Développé par OpenAI, publié en 2020.
- Modèle auto-régressif basé sur l'architecture Transformer.

Caractéristique	**Valeur / Détail**
Date	2020
Paramètres	175 milliards
Corpus d'entraînement	Environ 300 milliards de tokens
Contexte maximum	2048 tokens
Ressources GPU	Plusieurs centaines de GPU (type V100)
Coût d'entraînement (estim.)	4,6 M$ à 12 M$ (selon les sources)

- **Applications** : génération de texte, Q&R, résumé, traduction, assistance à la programmation.

Le few-shot learning

- Les grands modèles (comme GPT-3) peuvent réaliser des tâches **sans être explicitement réentraînés** dessus.
- Le principe repose sur **l'utilisation de quelques exemples** (exemples étiquetés) directement dans le prompt ou l'entrée.
- **Zero-shot** : aucune démonstration fournie, le modèle doit comprendre la tâche à partir de sa connaissance générale.
- **One-shot / Few-shot** : on inclut un nombre très réduit d'exemples (un ou quelques-uns) pour guider le modèle dans la résolution de la tâche.
- Exemple :

English: "cat" $\rightarrow$ French: "chat"
English: "house" $\rightarrow$ French: "maison"
English: "car" $\rightarrow$ French: "voiture"

English: "tree" $\rightarrow$ French:

ChatGPT

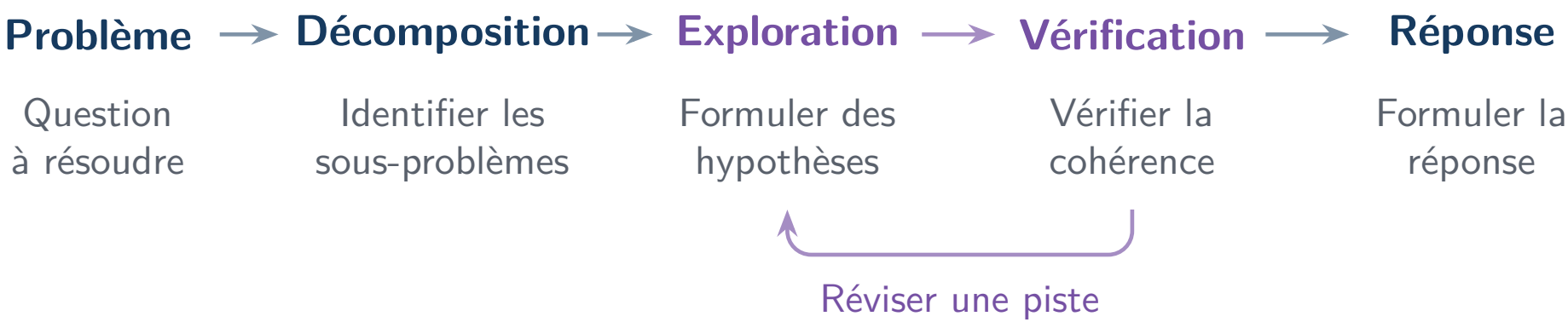
ChatGPT, développé par OpenAI, est un modèle de langage basé sur l'architecture GPT (Generative Pre-trained Transformer) optimisé pour comprendre et générer des dialogues naturels. L'entraînement de ChatGPT se décline selon les étapes

suivantes :

1. **Pré-entraînement sur un corpus volumineux :** Comme GPT-3, ChatGPT est d'abord pré-entraîné sur un vaste ensemble de données textuelles, englobant un large éventail de la littérature disponible sur Internet, pour apprendre une compréhension générale du langage.
2. **Fine-tuning supervisé :** Ensuite, ChatGPT est affiné sur des dialogues spécifiques pour améliorer ses compétences conversationnelles. Cette étape utilise des paires question-réponse et des conversations pour enseigner au modèle des structures de dialogue et des réponses contextuellement appropriées.
3. **Reinforcement Learning from Human Feedback (RLHF):** Utilisation de techniques de renforcement pour ajuster les réponses du modèle basées sur les préférences et les corrections fournies par des évaluateurs humains, raffinant davantage la pertinence et la naturalité des réponses.

Modèles de raisonnement

Ces modèles sont entraînés à décomposer un problème, explorer des pistes et vérifier leurs résultats avant de répondre.



Davantage de calcul peut améliorer la résolution des problèmes complexes, au prix d'un temps de réponse plus long.

LLMs propriétaires vs Open Source

- **Modèles propriétaires**
  - ChatGPT, Claude, Gemini, Grok...
  - Poids non disponibles au public.
  - Performances élevées, accès limité par API payante
  - Protection intellectuelle stricte (code non accessible)
  - Support technique assuré, documentation avancée
  - Dépendance aux fournisseurs et coûts potentiellement élevés

- **Modèles open source**
  - Llama, Mistral, Gemma, DeepSeek, Qwen...
  - Poids disponibles au public
  - Transparence du code, flexibilité d'adaptation
  - Performances parfois inférieures, mais amélioration continue
  - Gratuit, mais coût de l'infrastructure à gérer
  - Exposition aux failles potentielles de sécurité, support communautaire

Conclusion : Choisir entre contrôle, flexibilité et performances optimales.